ArduPilot PNT (Positioning, Navigation, Timing) Reference Audit

Read-only static-analysis audit — every snippet reproduced verbatim from the cited path:line range

Audited HEAD 5b67e27b0a · Core references: 63 · Indirect references: 31 · Total references: 94 · Evidence rows: 282 (main + Layer 1 + Layer 2)
Line-number basis: every cited path:line range in this document is stated as of the audited HEAD 5b67e27b0a (the pre-refactor baseline). The reusable-service extraction adds one additive, default-off set_update_dt timing seam to AP_L1_Control.cpp / .h after that commit; that seam relocates no behavior and only shifts those two files' line numbers, so the numbers here are intentionally not post-seam current-source lines.

Scope. This document is an audit-grade, read-only static-analysis catalog of every location in the ArduPilot firmware codebase (repository root containing ArduCopter/, ArduPlane/, Rover/, ArduSub/, AntennaTracker/, Blimp/, libraries/, modules/) where Positioning, Navigation, and Timing (PNT) data is handled, together with a two-layer dependency map for every catalogued reference. GROUP 1 catalogs core references — code that directly implements, reads, writes, or processes PNT data. GROUP 2 catalogs indirect / relational references — code that does not directly read or write PNT data but whose behavior changes as a result of PNT state. All file paths are repository-root-relative; every code snippet is reproduced VERBATIM from the cited path and line range so each row is independently verifiable. No source file is modified by this audit.

Executive Summary

This Executive Summary opens the audit and states up front what the catalog behind it establishes: where every Positioning, Navigation and Timing behavior in the ArduPilot firmware lives today, what depends on it, and — for the L1 lateral-navigation slice — which member of the reusable AfsimL1Behavior service each behavior is refactored into. It is a reading aid: the numbered tables, the instance mapping and the registers that follow remain the authoritative record.

Project Overview

This deliverable is an audit-grade, read-only static-analysis catalog of every location in the ArduPilot firmware codebase where Positioning, Navigation and Timing (PNT) data is handled, taken at audited HEAD 5b67e27b0a. Every catalogued reference reproduces the code at its cited path:line range verbatim and carries a two-layer dependency map: Layer 1 records the direct callers, callees and typed data edges around the reference, and Layer 2 records the full transitive chain through to the final consumer that acts on the value.

The catalog is extended with a current-location → new-service-location mapping. For every PNT touch-point inside the wrapped L1 guidance controller it names the member of the reusable AfsimL1Behavior service that the behavior is refactored into: the AHRS state shim that supplies injected position, velocity and attitude, the injected-dt timing seam that replaces the internal hardware clock, and the extern "C" command surface that returns the roll and lateral-acceleration commands. No source file is modified by this audit.

Core Problem Addressed

PNT behavior in ArduPilot is not a library that a host can call — it is embedded in the vehicle flight loop and duplicated across vehicles. Position, velocity, attitude and the control-step timebase are pulled implicitly from shared AHRS/EKF state and the hardware clock; PNT values arrive packed into structures alongside non-PNT fields; and the same derivation is repeated across the vehicle glue code. The catalog records that coupling directly: [SHARED-STRUCT] marks the packed structures, [DUPLICATED] marks the repeated derivations, and [CIRCULAR] marks the state that depends on itself.

The consequence is that an external host cannot reuse a PNT behavior without first knowing every place that behavior lives, what it reads, and what breaks if it moves. This audit establishes exactly that ground truth — an enumerated, independently verifiable inventory of every PNT touch-point together with its dependency neighbourhood — and, for the L1-navigation slice, names the service member each behavior moves to, so the extraction can be reviewed as a mapping between named locations rather than as a rewrite.

Key Stakeholders and Users

Integration and host engineers embedding the extracted guidance service use the current → new service location mapping as the wiring contract: it names, accessor by accessor, the value the host must inject and the service member that now supplies it. Flight-controls reviewers signing off the guidance seam use the Core Navigation and Core Timing tables, with their paired dependency sub-tables, to confirm that nothing outside the audited seam observes the values being injected.

Firmware maintainers auditing PNT coupling use the audit-discipline flags to locate circular state, shared structures and duplicated derivations before touching shared state, and the Coverage & Explicit-Absence Register to see which surfaces were swept and found clear. Reviewers and auditors rely on the read-only guarantee and on the verbatim path:line citations, which make every row independently checkable against the tree at the audited HEAD rather than something that has to be taken on trust.

Key Findings at a Glance

Every figure below is a reconciled total of the catalog itself, carried forward unchanged from the tables, the instance mapping and the registers that follow.

Dimension	Value	What the Catalog Establishes	Where It Is Catalogued
Core PNT references	63	Code that directly implements, reads, writes or processes PNT data.	GROUP 1 — CORE PNT REFERENCES: Table 1 Core Positioning, Table 2 Core Navigation, Table 3 Core Timing, each with its 1a / 2a / 3a dependency map.
Indirect / relational PNT references	31	Code that reads no PNT data itself but whose behavior changes as a result of PNT state.	GROUP 2 — INDIRECT / RELATIONAL PNT REFERENCES: Table 4 Indirect Positioning, Table 5 Indirect Navigation, Table 6 Indirect Timing, each with its 4a / 5a / 6a dependency map.
Total catalogued touch-points	94	Core plus indirect — the complete PNT surface enumerated by this audit.	All GROUP 1 and GROUP 2 main tables.
Evidence rows	282	94 touch-points × 3 layers: the main row, its Layer 1 direct edges and its Layer 2 transitive chain.	Each main table together with its matching Xa dependency sub-table.
Data objects	18	Catalogued table objects: 6 main tables plus their 6 Layer 1 and 6 Layer 2 blocks.	Tables 1 / 1a through 6 / 6a.
Catalog structure	12	Tables in 6 numbered groups: 6 main tables, each paired with an Xa sub-table holding its Layer 1 and Layer 2 blocks.	Tables 1 / 1a through 6 / 6a, rendered in that order.
Audit-discipline flags	5	Flag kinds, with occurrences [CIRCULAR] 8 · [SHARED-STRUCT] 19 · [DUPLICATED] 36 · [MULTI-CATEGORY] 7 · [ABSENT] 30. They mark extraction risk and enforce non-inference discipline inline in the Notes columns.	Notes columns throughout; taxonomy and occurrence counts in Audit-Discipline Flags.
Coverage and explicit absences	109	Register entries across 10 sub-registers, including 27 checked, explicitly documented absences — evidence that the sweep was exhaustive rather than opportunistic.	Coverage & Explicit-Absence Register.
Current → new service mapping	12	L1-navigation touch-points mapped onto the AfsimL1Behavior member each is refactored into, spanning position, velocity, attitude, airspeed scaling, timing, geometry and the command outputs.	PNT Instance Audit — Current Location → New Service Location.

Expected Impact and Value Proposition

The audit converts an implicit understanding of ArduPilot PNT coupling into a verifiable extraction baseline. Because every row cites a path:line range and reproduces the code at that range verbatim, the inventory can be re-checked against the tree at audited HEAD 5b67e27b0a rather than taken on trust, and the totals above reconcile against the catalog they summarise.

The audit-discipline flags turn extraction risk into explicit, addressable markers — where state is circular, where PNT fields are packed with non-PNT fields, where a derivation is duplicated, and, through the checked absences, where an expected PNT element was looked for and genuinely is not present. For the L1-navigation slice the current → new mapping supplies a documented wiring contract for the reusable service, so the guidance seam can be reviewed and signed off against named locations on both sides of the extraction.

Document Roadmap / How to Read This Document

The sections below appear in this order. Section names, table numbers and row numbering are exactly as catalogued.

Legend — The classification vocabularies — Role for Group 1, the Trigger Condition / PNT State Observed / Behavior Change discriminators for Group 2, Dependency Type for Layer 1 — plus the Notes-flag meanings and the cross-reference rule that ties each Xa table back to its main table numbering. Read this first.

Footprint Summary — The surveyed surface: the subsystems, receiver backends, EKF families and flight-mode files swept to make the catalog exhaustive.

GROUP 1 — CORE PNT REFERENCES — Tables 1 / 1a Core Positioning, 2 / 2a Core Navigation and 3 / 3a Core Timing: code that directly implements, reads, writes or processes PNT data. Table 2 also carries the New Service Location column.

GROUP 2 — INDIRECT / RELATIONAL PNT REFERENCES — Tables 4 / 4a Indirect Positioning, 5 / 5a Indirect Navigation and 6 / 6a Indirect Timing: code whose behavior changes as a result of PNT state.

PNT Instance Audit — Current Location → New Service Location — The consolidated mapping of every PNT touch-point inside the wrapped L1 controller onto its AfsimL1Behavior member, with the line-number basis stated alongside it.

Audit-Discipline Flags — The flag taxonomy with its occurrence counts, reconciled against the catalog.

Coverage & Explicit-Absence Register — The directory-wide token sweeps and checked absences that establish exhaustiveness across libraries/, the vehicle directories and the submodule boundary edges.

Provenance. This Executive Summary is derived entirely from the findings already catalogued in this document — the numbered tables, the instance mapping and the registers that follow — and no new static analysis of the ArduPilot codebase was performed to produce it. Every figure above is a reconciled total of that catalog, and every cited path:line reference in this document is unchanged: none was re-derived, re-scored or re-verified for this summary.

Legend

Role (Group 1)	Source = produces/writes PNT state from a measurement | Transform = fuses/derives/converts PNT state (EKF fusion, GPS blending, geodetic↔NED) | Sink = only reads PNT state to drive control/behavior.
Group 2 columns	Trigger Condition (what invokes the code) | PNT State Observed (the exact accessor/value read, e.g. gps.status(), ahrs.have_inertial_nav(), EKF variance) — the discriminator from Group 1 | Behavior Change Description (how behavior changes).
Dependency Type (Layer 1)	Data dependency (shared data value/struct field) | Function call (direct call) | Inheritance / Interface (backend overrides a virtual frontend interface) | Shared global / singleton (access via an AP:: accessor — AP::gps(), AP::ahrs(), AP::scheduler(), AP::rtc()).
Notes flags	[CIRCULAR] circular dependency (e.g. AHRS↔EKF) | [SHARED-STRUCT] PNT fields packed with non-PNT fields (extraction risk) | [DUPLICATED] copy-pasted PNT logic across locations | [MULTI-CATEGORY] spans >1 PNT category (emitted once per table) | [ABSENT] expected PNT element not found where checked.
Cross-reference	Each main table owns a monotonic, per-table # space (1,2,3…). Its dependency sub-table (the matching Xa table) reuses those exact numbers as Ref #. Each Xa table contains a Layer 1 block (direct edges) followed by a Layer 2 block (full transitive chain + final consumer + Chain Depth = the number of → hops explicitly listed in that chain).

Footprint Summary

Footprint surveyed: libraries/ = 151 subsystems; libraries/AP_GPS/*.{cpp,h} = 44 files (14 receiver backends each catalogued individually + AP_GPS_Blended + GPS_Backend base); EKF families AP_NavEKF/AP_NavEKF2/AP_NavEKF3 with AP_NavEKF3 split across 13 fusion modules (all catalogued); flight/drive-mode files = 81 (ArduCopter 29 + ArduPlane 26 + Rover 14 + ArduSub 12), enumerated in the Coverage & Explicit-Absence Register. PNT state is produced under libraries/ and consumed/acted-upon in vehicle glue code (failsafe, ekf_check, fence, flight-mode gating).

GROUP 1 — CORE PNT REFERENCES

Table 1 — Core Positioning

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes

1	libraries/AP_GPS/AP_GPS_UBLOX.cpp	1510-1518	AP_GPS_UBLOX::_parse_gps()(NAV-POSLLH)	Source	_last_pos_time = _buffer.posllh.itow;
					state.location.lng = _buffer.posllh.longitude;
					state.location.lat = _buffer.posllh.latitude;
					state.have_undulation = true;
					state.undulation = (_buffer.posllh.altitude_msl - _buffer.posllh.altitude_ellipsoid) * 0.001;
					set_alt_amsl_cm(state, _buffer.posllh.altitude_msl / 10);
					
					state.status = next_fix;
					_new_position = true;
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the u-blox UBX binary NAV-POSLLH. One of 14 receiver backends, each catalogued individually. target GPS_State.

2	libraries/AP_GPS/AP_GPS_NMEA.cpp	333-340	AP_GPS_NMEA::_term_complete()	Source	state.location.lat = _new_latitude;
					state.location.lng = _new_longitude;
					}
					state.num_sats = _new_satellite_count;
					state.hdop = _new_hdop;
					switch(_new_quality_indicator) {
					case 0: // Fix not available or invalid
					state.status = AP_GPS::NO_FIX;
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the NMEA-0183 ASCII (GGA/RMC) sentence. One of 14 receiver backends, each catalogued individually. target GPS_State.

3	libraries/AP_GPS/AP_GPS_SBF.cpp	484-491	AP_GPS_SBF::process_message()	Source	state.location.lat = (int32_t)(temp.Latitude * RAD_TO_DEG_DOUBLE * (double)1e7);
					state.location.lng = (int32_t)(temp.Longitude * RAD_TO_DEG_DOUBLE * (double)1e7);
					state.have_undulation = !is_DNU(temp.Undulation);
					double height = temp.Height; // in metres
					if (state.have_undulation) {
					height -= temp.Undulation;
					state.undulation = -temp.Undulation;
					}
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the Septentrio SBF binary PVTGeodetic. One of 14 receiver backends, each catalogued individually. target GPS_State.

4	libraries/AP_GPS/AP_GPS_NOVA.cpp	209-216	AP_GPS_NOVA::process_message()	Source	state.last_gps_time_ms = AP_HAL::millis();
					
					state.location.lat = (int32_t) (bestposu.lat * (double)1e7);
					state.location.lng = (int32_t) (bestposu.lng * (double)1e7);
					state.have_undulation = true;
					state.undulation = -bestposu.undulation;
					set_alt_amsl_cm(state, bestposu.hgt * 100);
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the NovAtel OEM BESTPOS. One of 14 receiver backends, each catalogued individually. target GPS_State.

5	libraries/AP_GPS/AP_GPS_SBP.cpp	276-283	AP_GPS_SBP::_attempt_state_update()	Source	
					state.location.lat = (int32_t) (pos_llh->lat * (double)1e7);
					state.location.lng = (int32_t) (pos_llh->lon * (double)1e7);
					state.location.alt = (int32_t) (pos_llh->height * 100);
					state.num_sats = pos_llh->n_sats;
					
					if (pos_llh->flags == 0) {
					state.status = AP_GPS::GPS_OK_FIX_3D;
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the Swift Navigation SBP v1 POS_LLH. One of 14 receiver backends, each catalogued individually. target GPS_State.

6	libraries/AP_GPS/AP_GPS_SBP2.cpp	328-335	AP_GPS_SBP2::_attempt_state_update()	Source	//
					state.location.lat = (int32_t) (last_pos_llh.lat * (double)1e7);
					state.location.lng = (int32_t) (last_pos_llh.lon * (double)1e7);
					state.location.alt = (int32_t) (last_pos_llh.height * 100);
					state.num_sats = last_pos_llh.n_sats;
					
					switch (last_pos_llh.flags.fix_mode) {
					case 1:
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the Swift Navigation SBP v2 POS_LLH. One of 14 receiver backends, each catalogued individually. target GPS_State.

7	libraries/AP_GPS/AP_GPS_SIRF.cpp	178-185	AP_GPS_SIRF::_parse_gps()	Source	state.status = AP_GPS::NO_FIX;
					}else if ((_buffer.nav.fix_type & FIX_MASK) == FIX_3D) {
					state.status = AP_GPS::GPS_OK_FIX_3D;
					}else{
					state.status = AP_GPS::GPS_OK_FIX_2D;
					}
					state.location.lat = int32_t(be32toh(_buffer.nav.latitude));
					state.location.lng = int32_t(be32toh(_buffer.nav.longitude));
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the SiRF binary geodetic navigation. One of 14 receiver backends, each catalogued individually. target GPS_State.

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes

8	libraries/AP_GPS/AP_GPS_ERB.cpp	150-157	AP_GPS_ERB::_parse_gps()	Source	`_last_pos_time = _buffer.pos.time;`
					`state.location.lng = (int32_t)(_buffer.pos.longitude * (double)1e7);`
					`state.location.lat = (int32_t)(_buffer.pos.latitude * (double)1e7);`
					`state.have_undulation = true;`
					`state.undulation = _buffer.pos.altitude_msl - _buffer.pos.altitude_ellipsoid;`
					`set_alt_amsl_cm(state, _buffer.pos.altitude_msl * 100);`
					`state.status = next_fix;`
					`_new_position = true;`
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the Emlid Reach ERB protocol. One of 14 receiver backends, each catalogued individually. target GPS_State.

9	libraries/AP_GPS/AP_GPS_MAV.cpp	67-75	AP_GPS_MAV::handle_msg()	Source	`state.status = (AP_GPS::GPS_Status)packet.fix_type;`
					
					`Location loc = {};`
					`loc.lat = packet.lat;`
					`loc.lng = packet.lon;`
					`if (have_alt) {`
					`    loc.alt = packet.alt * 100; // convert to centimeters`
					`}`
					`state.location = loc;`
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the MAVLink GPS_INPUT message. One of 14 receiver backends, each catalogued individually. target GPS_State.

10	libraries/AP_GPS/AP_GPS_MSP.cpp	44-51	AP_GPS_MSP::handle_msp()	Source	`state.status = (AP_GPS::GPS_Status)pkt.fix_type;`
					`state.num_sats = pkt.satellites_in_view;`
					`state.location = {`
					`    pkt.latitude,`
					`    pkt.longitude,`
					`    pkt.msl_altitude,`
					`    Location::AltFrame::ABSOLUTE`
					`};`
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the MSP protocol GPS data. One of 14 receiver backends, each catalogued individually. target GPS_State.

11	libraries/AP_GPS/AP_GPS_GSOF.cpp	231-238	AP_GPS_GSOF::pack_state_data()	Source	`state.location.lat = (int32_t)(RAD_TO_DEG_DOUBLE * position.latitude_rad * (double)1e7);`
					`state.location.lng = (int32_t)(RAD_TO_DEG_DOUBLE * position.longitude_rad * (double)1e7);`
					`state.location.alt = (int32_t)(position.altitude * 100);`
					`state.last_gps_time_ms = AP_HAL::millis();`
					
					`if ((vel.velocity_flags & 1) == 1) {`
					`    state.ground_speed = vel.horizontal_velocity;`
					`    state.ground_course = wrap_360(degrees(vel.heading));`
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the Trimble GSOF binary. One of 14 receiver backends, each catalogued individually. target GPS_State.

12	libraries/AP_GPS/AP_GPS_DroneCAN.cpp	388-396	AP_GPS_DroneCAN::handle_fix2_msg()	Source	`if (msg.height_msl_mm != msg.height_ellipsoid_mm) {`
					`    seen_valid_height_ellipsoid = true;`
					`}`
					`interim_state.have_undulation = seen_valid_height_ellipsoid;`
					`if (seen_valid_height_ellipsoid) {`
					`    interim_state.undulation = (msg.height_msl_mm - msg.height_ellipsoid_mm) * 0.001;`
					`}`
					`interim_state.location = loc;`
					`set_alt_amsl_cm(interim_state, alt_amsl_cm);`
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the DroneCAN/UAVCAN Fix2 (external boundary modules/DroneCAN). One of 14 receiver backends, each catalogued individually. target GPS_State.

13	libraries/AP_GPS/AP_GPS_ExternalAHRS.cpp	49-56	AP_GPS_ExternalAHRS::handle_external()	Source	`state.num_sats = pkt.satellites_in_view;`
					
					`const Location loc {`
					`    pkt.latitude,`
					`    pkt.longitude,`
					`    pkt.msl_altitude,`
					`    Location::AltFrame::ABSOLUTE`
					`};`
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the External AHRS/INS GPS feed. One of 14 receiver backends, each catalogued individually. target GPS_State.

14	libraries/AP_GPS/AP_GPS_SITL.cpp	86-93	AP_GPS_SITL::read()	Source	`state.status = AP_GPS::GPS_OK_FIX_3D;`
					`state.num_sats = 15;`
					
					`state.location = Location{`
					`    int32_t(latitude*1e7),`
					`    int32_t(longitude*1e7),`
					`    int32_t(altitude*100),`
					`    Location::AltFrame::ABSOLUTE`
						[SHARED-STRUCT] Receiver backend writes GPS_State.location/status from the Software-in-the-loop simulated GPS. One of 14 receiver backends, each catalogued individually. target GPS_State.

15	libraries/AP_GPS/GPS_Backend.cpp	101-109	AP_GPS_Backend::fill_3d_velocity()	Transform	`void AP_GPS_Backend::fill_3d_velocity(void)`
					`{`
					`    float gps_heading = radians(state.ground_course);`
					
					`    state.velocity.x = state.ground_speed * cosf(gps_heading);`
					`    state.velocity.y = state.ground_speed * sinf(gps_heading);`
					`    state.velocity.z = 0;`
					`    state.have_vertical_velocity = false;`
					`}`
						Backend base class; derives state.velocity (NED) from ground_speed + ground_course for receivers that do not report 3D velocity. Shared by all backends.

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes

16	libraries/AP_GPS/AP_GPS.h	324-329	AP_GPS::location()	Source	const Location &location(uint8_t instance) const {
					    return state[instance].location;
					}
					const Location &location() const {
					    return location(primary_instance);
					}
						Frontend accessor exposing backend-measured GPS_State.location; dispatches to primary_instance. Reached vehicle-side via the AP::gps() singleton.

17	libraries/AP_GPS/AP_GPS.h	360-365	AP_GPS::velocity()	Source	const Vector3f &velocity(uint8_t instance) const {
					    return state[instance].velocity;
					}
					const Vector3f &velocity() const {
					    return velocity(primary_instance);
					}
						[MULTI-CATEGORY] 3D NED velocity accessor of measured GPS_State.velocity. velocity also drives Navigation fusion (Table 2).

18	libraries/AP_GPS/AP_GPS.h	192-201	structAP_GPS::GPS_State	Source	struct GPS_State {
					    uint8_t instance; // the instance number of this GPS
					
					    // all the following fields must all be filled by the backend driver
					    GPS_Status status; ///< driver fix status
					    uint32_t time_week_ms; ///< GPS time (milliseconds from start of GPS week)
					    uint16_t time_week; ///< GPS week number
					    Location location; ///< last fix location
					    float ground_speed; ///< ground speed in m/s
					    float ground_course; ///< ground course in degrees, wrapped 0-360
						[SHARED-STRUCT] [MULTI-CATEGORY] One struct mixes Positioning (location), Navigation (velocity, ground_speed, ground_course) and Timing (time_week_ms, time_week) — see Table 3 #8. Primary extraction-risk struct for a PNT split.

19	libraries/AP_GPS/AP_GPS_Blended.cpp	17-25	AP_GPS_Blended::_calc_weights()	Transform	bool AP_GPS_Blended::_calc_weights(void)
					{
					    // zero the blend weights
					    memset(&_blend_weights, 0, sizeof(_blend_weights));
					
					    static_assert(GPS_MAX_RECEIVERS == 2, "GPS blending only currently works with 2 receivers");
					    // Note that the early quit below relies upon exactly 2 instances
					    // The time delta calculations below also rely upon every instance being currently detected and being parsed
						Multi-receiver blending of location/velocity (GPS_MAX_RECEIVERS==2). Reads gps.state[].status (Data dependency) to weight receivers.

20	libraries/AP_Common/Location.h	11-20	classLocation	Source	uint8_t relative_alt : 1; // 1 if altitude is relative to home
					uint8_t loiter_ccw : 1; // 0 if clockwise, 1 if counter clockwise
					uint8_t terrain_alt : 1; // this altitude is above terrain
					uint8_t origin_alt : 1; // this altitude is above ekf origin
					uint8_t loiter_xtrack : 1; // 0 to crosstrack from center of waypoint, 1 to crosstrack from tangent exit location
					
					// note that mission storage only stores 24 bits of altitude (~ +/- 83km)
					int32_t alt; // in cm
					int32_t lat; // in 1E7 degrees
					int32_t lng; // in 1E7 degrees
						[SHARED-STRUCT] Geodetic PNT fields lat/lng (1E7 deg) and alt (cm) packed with NON-PNT bitfields relative_alt, loiter_ccw, terrain_alt, origin_alt, loiter_xtrack. Ubiquitous value type; top extraction risk.

21	libraries/AP_Common/Location.cpp	259-268	Location::get_vector_from_origin_NEU_cm()	Transform	bool Location::get_vector_from_origin_NEU_cm(T &vec_neu) const
					{
					    // convert altitude
					    int32_t alt_above_origin_cm = 0;
					    if (!get_alt_cm(AltFrame::ABOVE_ORIGIN, alt_above_origin_cm)) {
					        return false;
					    }
					
					    // convert lat, lon
					    if (!get_vector_xy_from_origin_NE_cm(vec_neu.xy())) {
						Geodetic (lat/lng/alt) NEU centimetre vector relative to → the EKF origin (geodetic↔NED duality).

22	libraries/AP_InertialNav/AP_InertialNav.cpp	14-23	AP_InertialNav::update()	Transform	void AP_InertialNav::update(bool high_vibes)
					{
					    // get the NE position relative to the local earth frame origin
					    Vector2f posNE;
					    if (_ahrs_ekf.get_relative_position_NE_origin_float(posNE)) {
					        _relpos_cm.x = posNE.x * 100; // convert from m to cm
					        _relpos_cm.y = posNE.y * 100; // convert from m to cm
					    }
					
					    // get the D position relative to the local earth frame origin
						Pulls EKF origin-relative NE/D position into _relpos_cm (cm). get_position_neu_cm() exposes it to vehicle code.

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes

23	libraries/AC_AttitudeControl/AC_PosControl.cpp	363-372	AC_PosControl::input_pos_NEU_cm()	Sink	void AC_PosControl::input_pos_NEU_cm(const Vector3p& pos_neu_cm, float pos_terrain_target_u_cm, float terrain_buffer_cm)
					{
					input_pos_NEU_m(pos_neu_cm * 0.01, pos_terrain_target_u_cm * 0.01, terrain_buffer_cm * 0.01);
					}
					
					// Sets a new NEU position target in meters and computes a jerk-limited trajectory.
					// Updates internal acceleration commands using a smooth kinematic path constrained
					// by configured acceleration and jerk limits. Terrain margin is used to constrain
					// horizontal velocity to avoid vertical buffer violation.
					void AC_PosControl::input_pos_NEU_m(const Vector3p& pos_neu_m, float pos_terrain_target_u_m, float terrain_buffer_m)
						[ABSENT] 3D position/velocity controller consuming the position target. there is no standalone AC_PosControl library — it physically resides under libraries/AC_AttitudeControl/ (snapshot-fidelity note).

24	libraries/APM_Control/AR_PosControl.cpp	114-123	AR_PosControl::update()	Sink	void AR_PosControl::update(float dt)
					{
					// exit immediately if no current location, destination or disarmed
					Vector3p curr_pos_NED_m;
					Vector3f curr_vel_NED;
					if (!hal.util->get_soft_armed() ||
					!AP::ahrs().get_relative_position_NED_origin(curr_pos_NED_m) ||
					!AP::ahrs().get_velocity_NED(curr_vel_NED)) {
					_desired_speed = _atc.get_desired_speed_accel_limited(0.0f, dt);
					_desired_lat_accel = 0.0f;
					_desired_turn_rate_rads = 0.0f;
						Rover position controller; reads AP::ahrs().get_relative_position_NED_origin() and get_velocity_NED() (Shared global/singleton). Separate controller from Copter's AC_PosControl.

25	libraries/AP_Beacon/AP_Beacon.cpp	175-184	AP_Beacon::get_vehicle_position_ned()	Source	bool AP_Beacon::get_vehicle_position_ned(Vector3f &position, float& accuracy_estimate) const
					{
					if (!device_ready()) {
					return false;
					}
					
					// check for timeout
					if (AP_HAL::millis() - veh_pos_update_ms > AP_BEACON_TIMEOUT_MS) {
					return false;
					}
						Non-GPS (GPS-denied) positioning source; returns vehicle NED position with a staleness timeout.

26	libraries/AP_VisualOdom/AP_VisualOdom.cpp	201-210	AP_VisualOdom::handle_pose_estimate()	Source	void AP_VisualOdom::handle_pose_estimate(uint64_t remote_time_us, uint32_t time_ms, float x, float y, float z, float roll, float pitch, float yaw, float posErr, float angErr, uint8_t reset_counter, int8_t quality)
					{
					// exit immediately if not enabled
					if (!enabled()) {
					return;
					}
					
					// call backend
					if (_driver != nullptr) {
					// convert attitude to quaternion and call backend
						Non-GPS visual-odometry position/attitude source feeding EKF external-nav fusion.

Table 1a — Dependency Map (Layer 1 direct edges + Layer 2 transitive chains)

Table 1a: Layer 1 — direct callers / callees with typed dependency

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	AP_GPS_UBLOX backend	writes GPS_State.location/status; fill_3d_velocity()	AP_GPS::update() backend read() →	Inheritance/Interface	_last_pos_time = _buffer.posllh.itow;
					state.location.lng = _buffer.posllh.longitude;
					state.location.lat = _buffer.posllh.latitude;
					state.have_undulation = true;
					state.undulation = (_buffer.posllh.altitude_msl - _buffer.posllh.altitude_ellipsoid) * 0.001;
					set_alt_amsl_cm(state, _buffer.posllh.altitude_msl / 10);
					state.status = next_fix;
					_new_position = true;

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

2	AP_GPS_NMEA backend	writes GPS_State.location/status;
		fill_3d_velocity()
			AP_GPS::update() backend read() → Page	Inheritance/Interface	`state.location.lat = _new_latitude;`
					`state.location.lng = _new_longitude;`
					`}`
					`state.num_sats = _new_satellite_count;`
					`state.hdop = _new_hdop;`
					`switch(_new_quality_indicator) {`
					`case 0: // Fix not available or invalid`
					`state.status = AP_GPS::NO_FIX;`

3	AP_GPS_SBF backend	writes GPS_State.location/status;
		fill_3d_velocity()
			AP_GPS::update() backend read() →	Inheritance/Interface	`state.location.lat = (int32_t)(temp.Latitude * RAD_TO_`
					`DEG_DOUBLE * (double)1e7);`
					`state.location.lng = (int32_t)(temp.Longitude * RAD_TO`
					`_DEG_DOUBLE * (double)1e7);`
					`state.have_undulation = !is_DNU(temp.Undulation);`
					`double height = temp.Height; // in metres`
					`if (state.have_undulation) {`
					`height -= temp.Undulation;`
					`state.undulation = -temp.Undulation;`
					`}`

4	AP_GPS_NOVA backend	writes GPS_State.location/status;
		fill_3d_velocity()
			AP_GPS::update() backend read() →	Inheritance/Interface	`state.last_gps_time_ms = AP_HAL::millis();`
					`state.location.lat = (int32_t) (bestposu.lat * (double)1e7`
					`);`
					`state.location.lng = (int32_t) (bestposu.lng * (double)1e7`
					`);`
					`state.have_undulation = true;`
					`state.undulation = -bestposu.undulation;`
					`set_alt_amsl_cm(state, bestposu.hgt * 100);`

5	AP_GPS_SBP backend	writes GPS_State.location/status;
		fill_3d_velocity()
			AP_GPS::update() backend read() →	Inheritance/Interface	`state.location.lat = (int32_t) (pos_llh->lat * (doubl`
					`e)1e7);`
					`state.location.lng = (int32_t) (pos_llh->lon * (doubl`
					`e)1e7);`
					`state.location.alt = (int32_t) (pos_llh->height * 100`
					`);`
					`state.num_sats = pos_llh->n_sats;`
					`if (pos_llh->flags == 0) {`
					`state.status = AP_GPS::GPS_OK_FIX_3D;`

6	AP_GPS_SBP2 backend	writes GPS_State.location/status;
		fill_3d_velocity()
			AP_GPS::update() backend read() →	Inheritance/Interface	`//`
					`state.location.lat = (int32_t) (last_pos_llh.lat * (d`
					`ouble)1e7);`
					`state.location.lng = (int32_t) (last_pos_llh.lon * (d`
					`ouble)1e7);`
					`state.location.alt = (int32_t) (last_pos_llh.height *`
					`100);`
					`state.num_sats = last_pos_llh.n_sats;`
					`switch (last_pos_llh.flags.fix_mode) {`
					`case 1:`

7	AP_GPS_SIRF backend	writes GPS_State.location/status;
		fill_3d_velocity()
			AP_GPS::update() backend read() →	Inheritance/Interface	`state.status = AP_GPS::NO_FIX;`
					`}else if ((_buffer.nav.fix_type & FIX_MASK) == FIX_3D) {`
					`state.status = AP_GPS::GPS_OK_FIX_3D;`
					`}else{`
					`state.status = AP_GPS::GPS_OK_FIX_2D;`
					`}`
					`state.location.lat = int32_t(be32toh(_buffer.nav.lati`
					`tude));`
					`state.location.lng = int32_t(be32toh(_buffer.nav.long`
					`itude));`

8	AP_GPS_ERB backend	writes GPS_State.location/status;
		fill_3d_velocity()
			AP_GPS::update() backend read() → Page	Inheritance/Interface	`_last_pos_time = _buffer.pos.time;`
					`state.location.lng = (int32_t)(_buffer.pos.longitude *`
					`(double)1e7);`
					`state.location.lat = (int32_t)(_buffer.pos.latitude * (`
					`double)1e7);`
					`state.have_undulation = true;`
					`state.undulation = _buffer.pos.altitude_msl - _buffer.pos.`
					`altitude_ellipsoid;`
					`set_alt_amsl_cm(state, _buffer.pos.altitude_msl * 100);`
					`state.status = next_fix;`
					`_new_position = true;`

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

9	AP_GPS_MAV backend	writes GPS_State.location/status; fill_3d_velocity()	AP_GPS::update() backend read() →	Inheritance/Interface	`state.status = (AP_GPS::GPS_Status)packet.fix_type;`
					`Location loc = {};`
					`loc.lat = packet.lat;`
					`loc.lng = packet.lon;`
					`if (have_alt) {`
					`loc.alt = packet.alt * 100; // convert to centimet`
					`ers`
					`}`
					`state.location = loc;`

10	AP_GPS_MSP backend	writes GPS_State.location/status; fill_3d_velocity()	AP_GPS::update() backend read() →	Inheritance/Interface	`state.status = (AP_GPS::GPS_Status)pkt.fix_type;`
					`state.num_sats = pkt.satellites_in_view;`
					`state.location = {`
					`pkt.latitude,`
					`pkt.longitude,`
					`pkt.msl_altitude,`
					`Location::AltFrame::ABSOLUTE`
					`};`

11	AP_GPS_GSOF backend	writes GPS_State.location/status; fill_3d_velocity()	AP_GPS::update() backend read() →	Inheritance/Interface	`state.location.lat = (int32_t)(RAD_TO_DEG_DOUBLE * position.la`
					`titude_rad * (double)1e7);`
					`state.location.lng = (int32_t)(RAD_TO_DEG_DOUBLE * position.lo`
					`ngitude_rad * (double)1e7);`
					`state.location.alt = (int32_t)(position.altitude * 100);`
					`state.last_gps_time_ms = AP_HAL::millis();`
					`if ((vel.velocity_flags & 1) == 1) {`
					`state.ground_speed = vel.horizontal_velocity;`
					`state.ground_course = wrap_360(degrees(vel.heading));`

12	AP_GPS_DroneCAN backend	writes GPS_State.location/status; fill_3d_velocity()	AP_GPS::update() backend read() →	Inheritance/Interface	`if (msg.height_msl_mm != msg.height_ellipsoid_mm) {`
					`seen_valid_height_ellipsoid = true;`
					`}`
					`interim_state.have_undulation = seen_valid_height_ellipsoi`
					`d;`
					`if (seen_valid_height_ellipsoid) {`
					`interim_state.undulation = (msg.height_msl_mm - msg.he`
					`ight_ellipsoid_mm) * 0.001;`
					`}`
					`interim_state.location = loc;`
					`set_alt_amsl_cm(interim_state, alt_amsl_cm);`

13	AP_GPS_ExternalAHRS backend	writes GPS_State.location/status; fill_3d_velocity()	AP_GPS::update() backend read() →	Inheritance/Interface	`state.num_sats = pkt.satellites_in_view;`
					`const Location loc {`
					`pkt.latitude,`
					`pkt.longitude,`
					`pkt.msl_altitude,`
					`Location::AltFrame::ABSOLUTE`
					`};`

14	AP_GPS_SITL backend	writes GPS_State.location/status; fill_3d_velocity()	AP_GPS::update() backend read() →	Inheritance/Interface	`state.status = AP_GPS::GPS_OK_FIX_3D;`
					`state.num_sats = 15;`
					`state.location = Location{`
					`int32_t(latitude*1e7),`
					`int32_t(longitude*1e7),`
					`int32_t(altitude*100),`
					`Location::AltFrame::ABSOLUTE`

15	AP_GPS_Backend::fill_3d_velocity ()	cosf/sinf on state.ground_course; writes state.velocity	All receiver backends (UBLOX, NMEA, SIRF, ...) Page	Data dependency	`void AP_GPS_Backend::fill_3d_velocity(void)`
					`{`
					`float gps_heading = radians(state.ground_course);`
					`state.velocity.x = state.ground_speed * cosf(gps_heading);`
					`state.velocity.y = state.ground_speed * sinf(gps_heading);`
					`state.velocity.z = 0;`
					`state.have_vertical_velocity = false;`
					`}`

16	AP_GPS::location()	returns state[instance].location	NavEKF3_Measurements::readGpsData (); AP_AHRS; AC_WPNav	Function call	`const Location &location(uint8_t instance) const {`
					`return state[instance].location;`
					`}`
					`const Location &location() const {`
					`return location(primary_instance);`
					`}`

17	AP_GPS::velocity()	returns state[instance].velocity	NavEKF3_Measurements::readGpsData () (PosVel fusion)	Function call	`const Vector3f &velocity(uint8_t instance) const {`
					`return state[instance].velocity;`
					`}`
					`const Vector3f &velocity() const {`
					`return velocity(primary_instance);`
					`}`

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

18	AP_GPS::GPS_State (struct)	(data struct — no calls)	All GPS backends (write); frontend accessors (read)	Data dependency	struct GPS_State {
					uint8_t instance; // the instance number of this GPS
					// all the following fields must all be filled by the back
					end driver
					GPS_Status status; ///< driver fix status
					uint32_t time_week_ms; ///< GPS time (millise
					conds from start of GPS week)
					uint16_t time_week; ///< GPS week number
					Location location; ///< last fix location
					float ground_speed; ///< ground speed in m
					/s
					float ground_course; ///< ground course in
					degrees, wrapped 0-360

19	AP_GPS_Blended::_calc_weights( )	gps.state[].status; time deltas	AP_GPS_Blended::calc_weights()/updat e (blended primary)	Data dependency	bool AP_GPS_Blended::_calc_weights(void)
					{
					// zero the blend weights
					memset(&_blend_weights, 0, sizeof(_blend_weights));
					static_assert(GPS_MAX_RECEIVERS == 2, "GPS blending only curre
					ntly works with 2 receivers");
					// Note that the early quit below relies upon exactly 2 instan
					ces
					// The time delta calculations below also rely upon every inst
					ance being currently detected and being parsed

20	class Location	get_alt_cm();
		get_vector_xy_from_origin_NE_cm()
			GPS_State; AP_AHRS; AC_WPNav; AP_Mission (ubiquitous)	Data dependency	uint8_t relative_alt : 1; // 1 if altitude is relati
					ve to home
					uint8_t loiter_ccw : 1; // 0 if clockwise, 1 if co
					unter clockwise
					uint8_t terrain_alt : 1; // this altitude is above
					terrain
					uint8_t origin_alt : 1; // this altitude is above
					ekf origin
					uint8_t loiter_xtrack : 1; // 0 to crosstrack from ce
					nter of waypoint, 1 to crosstrack from tangent exit location
					// note that mission storage only stores 24 bits of altitude (
					~ +/- 83km)
					int32_t alt; // in cm
					int32_t lat; // in 1E7 degrees
					int32_t lng; // in 1E7 degrees

21	Location::get_vector_from_origin_ NEU_cm()	get_alt_cm();
		get_vector_xy_from_origin_NE_cm()
			AC_WPNav::get_vector_NEU_cm() Page	Function call	bool Location::get_vector_from_origin_NEU_cm(T &vec_neu) const
					{
					// convert altitude
					int32_t alt_above_origin_cm = 0;
					if (!get_alt_cm(AltFrame::ABOVE_ORIGIN, alt_above_origin_cm))
					{
					return false;
					}
					// convert lat, lon
					if (!get_vector_xy_from_origin_NE_cm(vec_neu.xy())) {

22	AP_InertialNav::update()	_ahrs_ekf.get_relative_position_NE_ori gin_float()	Copter::read_inertia()	Function call	void AP_InertialNav::update(bool high_vibes)
					{
					// get the NE position relative to the local earth frame origi
					n
					Vector2f posNE;
					if (_ahrs_ekf.get_relative_position_NE_origin_float(posNE)) {
					_relpos_cm.x = posNE.x * 100; // convert from m to cm
					_relpos_cm.y = posNE.y * 100; // convert from m to cm
					}
					// get the D position relative to the local earth frame origin

23	AC_PosControl::input_pos_NEU_c m()	input_pos_NEU_m()	ArduCopter mode_auto.cpp / mode.cpp	Function call	void AC_PosControl::input_pos_NEU_cm(const Vector3p& pos_neu_cm, f
					loat pos_terrain_target_u_cm, float terrain_buffer_cm)
					{
					input_pos_NEU_m(pos_neu_cm * 0.01, pos_terrain_target_u_cm * 0
					.01, terrain_buffer_cm * 0.01);
					}
					// Sets a new NEU position target in meters and computes a jerk-li
					mited trajectory.
					// Updates internal acceleration commands using a smooth kinematic
					path constrained
					// by configured acceleration and jerk limits. Terrain margin is u
					sed to constrain
					// horizontal velocity to avoid vertical buffer violation.
					void AC_PosControl::input_pos_NEU_m(const Vector3p& pos_neu_m, flo
					at pos_terrain_target_u_m, float terrain_buffer_m)

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet
24	AR_PosControl::update()	AP::ahrs().get_relative_position_NED_origin(); get_velocity_NED()	Rover mode controllers	Shared global/singleton	`void AR_PosControl::update(float dt) { // exit immediately if no current location, destination or disarmed Vector3p curr_pos_NED_m; Vector3f curr_vel_NED; if (!hal.util->get_soft_armed() || !AP::ahrs().get_relative_position_NED_origin(curr_pos_NED_m) || !AP::ahrs().get_velocity_NED(curr_vel_NED)) { _desired_speed = _atc.get_desired_speed_accel_limited(0.0f , dt); _desired_lat_accel = 0.0f; _desired_turn_rate_rads = 0.0f;`
25	AP_Beacon::get_vehicle_position_ned()	device_ready(); AP_HAL::millis()	NavEKF3 range-beacon fusion	Function call	`bool AP_Beacon::get_vehicle_position_ned(Vector3f &position, float& accuracy_estimate) const { if (!device_ready()) { return false; } // check for timeout if (AP_HAL::millis() - veh_pos_update_ms > AP_BEACON_TIMEOUT_MS) { return false; }`
26	AP_VisualOdom::handle_pose_estimate()	_driver backend handle_pose_estimate()	GCS / AP_ExternalAHRS pose handlers Page	Inheritance/Interface	`void AP_VisualOdom::handle_pose_estimate(uint64_t remote_time_us, uint32_t time_ms, float x, float y, float z, float roll, float pitch, float yaw, float posErr, float angErr, uint8_t reset_counter, int8_t quality) { // exit immediately if not enabled if (!enabled()) { return; } // call backend if (_driver != nullptr) { // convert attitude to quaternion and call backend`

Table 1a: Layer 2 — full transitive chain, final consumer & hop depth

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	AP_GPS_UBLOX (state.location)	`AP_GPS_UBLOX → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
2	AP_GPS_NMEA (state.location)	`AP_GPS_NMEA → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
3	AP_GPS_SBF (state.location)	`AP_GPS_SBF → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
4	AP_GPS_NOVA (state.location)	`AP_GPS_NOVA → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
5	AP_GPS_SBP (state.location)	`AP_GPS_SBP → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
6	AP_GPS_SBP2 (state.location)	`AP_GPS_SBP2 → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
7	AP_GPS_SIRF (state.location)	`AP_GPS_SIRF → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
8	AP_GPS_ERB (state.location)	`AP_GPS_ERB → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
9	AP_GPS_MAV (state.location)	`AP_GPS_MAV → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
10	AP_GPS_MSP (state.location)	`AP_GPS_MSP → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
11	AP_GPS_GSOF (state.location)	`AP_GPS_GSOF → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
12	AP_GPS_DroneCAN (state.location)	`AP_GPS_DroneCAN → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
13	AP_GPS_ExternalAHRS (state.location)	`AP_GPS_ExternalAHRS → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
14	AP_GPS_SITL (state.location)	`AP_GPS_SITL → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors / SRV_Channel	8	Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED.
15	GPS velocity (fill_3d_velocity)	`GPS_State.velocity → AP_GPS::velocity() → NavEKF3_Measurements::readGpsData → NavEKF3::FuseVelPosNED → AP_AHRS::get_velocity_NED() → AC_PosControl → AP_Motors`	AP_Motors	6	[MULTI-CATEGORY] also Navigation.
16	AP_GPS::location()	`AP_GPS::location() → NavEKF3_Measurements::readGpsData (gps.location) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNav::set_wp_destination_loc → AC_PosControl → AP_Motors Page 11`	AP_Motors	6	Geodetic position path (ingestion hop explicit).
17	AP_GPS::velocity()	`AP_GPS::velocity() → NavEKF3_Measurements::readGpsData (gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_velocity_NED() → AC_PosControl → AP_Motors`	AP_Motors	5	Velocity path (ingestion hop explicit).
18	GPS_State (struct)	`GPS_State → AP_GPS accessors {location|velocity|time_week} → NavEKF3_Measurements → EKF/AHRS + AP_RTC → controllers + clock`	AP_Motors and AP_RTC	4	[SHARED-STRUCT] [MULTI-CATEGORY] branches into Table 3.
19	AP_GPS_Blended	`Blended → AP_GPS primary (blended) location/velocity → NavEKF3_Measurements → NavEKF3::FuseVelPosNED → AP_AHRS → controllers → AP_Motors`	AP_Motors	6	Transform upstream of EKF ingestion.
20	class Location (geodetic value type)	`Location → AC_WPNav::set_wp_destination_loc → get_vector_NEU_cm → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors	5	[SHARED-STRUCT] Pervasive value type; depth measured along its dominant waypoint-control consumer branch (the AC_WPNav→PosControl path).
21	Location→NEU conversion	`get_vector_from_origin_NEU_cm() → AC_WPNav::get_vector_NEU_cm → set_wp_destination_NEU_cm → AC_PosControl`	AC_PosControl	3	Geodetic↔NED transform edge.
22	AP_InertialNav	`EKF origin-relative pos → AP_InertialNav::update → _relpos_cm → Copter logging/reporting`	GCS / logger	3	Reporting branch.
23	AC_PosControl (Sink)	`AP_AHRS pos → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors`	AP_Motors	4	Inbound chain to a Sink endpoint.
24	AR_PosControl (Sink, Rover)	`AP_AHRS pos/vel → AR_PosControl → AR_AttitudeControl → SRV_Channels`	SRV_Channels	3	Rover actuation endpoint.
25	AP_Beacon (Source)	`AP_Beacon → get_vehicle_position_ned → NavEKF3 RngBcn fusion → AP_AHRS → AP_Motors`	AP_Motors	4	GPS-denied source.
26	AP_VisualOdom (Source)	`AP_VisualOdom → NavEKF3 external-nav fusion → AP_AHRS → AP_Motors Page 12`	AP_Motors	3	GPS-denied source.

Table 2 — Core Navigation

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes	New Service Location

1	libraries/AP_AHRS/AP_AHRS.cpp	3600-3604	AP_AHRS::get_location()	Transform	`bool AP_AHRS::get_location(Location &loc) const`
					`{`
					`    loc = state.location;`
					`    return state.location_ok;`
					`}`
						[CIRCULAR] Central navigation hub returns the fused Location from the active EKF. Reached via AP::ahrs(). AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference.	—

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes	New Service Location

2	libraries/AP_AHRS/AP_AHRS.h	238-246	AP_AHRS::groundspeed()	Sink	// EKF has a better ground speed vector estimate
					const Vector2f &groundspeed_vector() const { return state.ground_speed_vec; }
					
					// return ground speed estimate in meters/second. Used by ground vehicles.
					float groundspeed(void) const { return state.ground_speed; }
					
					const Vector3f &get_accel_ef() const {
					return state.accel_ef;
					}
						Read accessor returning the already-fused state.ground_speed (m/s). Performs no fusion and produces nothing from a measurement — classified Sink (read accessor), NOT Source.	—

3	libraries/AP_AHRS/AP_AHRS.h	269-277	AP_AHRS::get_origin()/have_inertial_nav()	Sink	// returns the inertial navigation origin in lat/lon/alt
					bool get_origin(Location &ret) const WARN_IF_UNUSED;
					
					bool have_inertial_nav() const;
					
					// return a ground velocity in meters/second, North/East/Down
					// order. Must only be called if have_inertial_nav() is true
					bool get_velocity_NED(Vector3f &vec) const WARN_IF_UNUSED;
						Availability/origin read accessors (have_inertial_nav() is a user-cited symbol, L273). They report EKF availability/origin — read accessors, NOT measurement Sources; classified Sink. EKF origin is WRITTEN by the EKF (see #15 calcGpsGoodToAlign / setOrigin).	—

4	libraries/AP_AHRS/AP_AHRS.cpp	1739-1748	AP_AHRS::get_relative_position_NED_origin()	Transform	bool AP_AHRS::get_relative_position_NED_origin(Vector3p &vec) const
					{
					switch (active_EKF_type()) {
					#if AP_AHRS_DCM_ENABLED
					case EKFType::DCM:
					return dcm.get_relative_position_NED_origin(vec);
					#endif
					#if HAL_NAVEKF2_AVAILABLE
					case EKFType::TWO: {
					Vector2p posNE;
						Geodetic↔NED: origin-relative NED position via active-EKF-type dispatch (DCM/EKF2/EKF3) — performs conversion on read.	—

5	libraries/AP_AHRS/AP_AHRS.cpp	3662-3666	AP_AHRS::get_velocity_NED()	Transform	bool AP_AHRS::get_velocity_NED(Vector3f &vec) const
					{
					vec = state.velocity_NED;
					return state.velocity_NED_ok;
					}
						Exposes the fused NED velocity (nav-hub output of EKF fusion); consumed by crash detection and Rover position control.	—

6	libraries/AP_NavEKF3/AP_NavEKF3_core.cpp	627-636	NavEKF3_core::UpdateFilter()	Transform	void NavEKF3_core::UpdateFilter(bool predict)
					{
					// don't run filter updates if states have not been initialised
					if (!statesInitialised) {
					return;
					}
					
					fill_scratch_variables();
					
					// update sensor selection (for affinity)
						[CIRCULAR] Extended Kalman Filter predict/fuse step (the core PNT transform). AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference. One of 13 AP_NavEKF3 fusion modules, all catalogued.	—

7	libraries/AP_NavEKF3/AP_NavEKF3_PosVelFusion.cpp	502-511	NavEKF3_core::SelectVelPosFusion()	Transform	void NavEKF3_core::SelectVelPosFusion()
					{
					// Check if the magnetometer has been fused on that time step and the filter is running at faster than 200 Hz
					// If so, don't fuse measurements on this time step to reduce frame over-runs
					// Only allow one time slip to prevent high rate magnetometer data preventing fusion of other measurements
					if (magFusePerformed && dtIMUavg < 0.005f && !posVelFusionDelayed) {
					posVelFusionDelayed = true;
					return;
					} else {
					posVelFusionDelayed = false;
						[CIRCULAR] GPS position/velocity fusion scheduling; the dtIMUavg<0.005 gate couples EKF fusion cadence to the scheduler loop rate (timing coupling). AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference. One of 13 AP_NavEKF3 fusion modules, all catalogued.	—

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes	New Service Location

8	libraries/AP_NavEKF3/AP_NavEKF3_PosVelFusion.cpp	694-702	NavEKF3_core::FuseVelPosNED()	Transform	void NavEKF3_core::FuseVelPosNED()
					{
					    // declare variables used to control access to arrays
					    bool fuseData[6] {};
					    uint8_t stateIndex;
					    uint8_t obsIndex;
					
					    // declare variables used by state and covariance update calculations
					    Vector6 R_OBS; // Measurement variances used for fusion
						Core position/velocity measurement-update (Kalman gain applied to GPS/baro NED innovations). One of 13 AP_NavEKF3 fusion modules, all catalogued.	—

9	libraries/AP_NavEKF3/AP_NavEKF3_Measurements.cpp	559-567	NavEKF3_core::readGpsData()	Transform	void NavEKF3_core::readGpsData()
					{
					    // check for new GPS data
					    const auto &gps = dal.gps();
					
					    // limit update rate to avoid overflowing the FIFO buffer
					    if (gps.last_message_time_ms(selected_gps) - lastTimeGpsReceived_ms <= frontend->sensorIntervalMin_ms) {
					        return;
					    }
						[CIRCULAR] GPS ingestion into the EKF delayed-time buffer (reads gps.location()/velocity()/status via AP_DAL). This is the GPS→EKF hop omitted earlier (cf. Table 1a chains). AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference. One of 13 AP_NavEKF3 fusion modules, all catalogued.	—

10	libraries/AP_NavEKF3/AP_NavEKF3_Control.cpp	232-240	NavEKF3_core::setAidingMode()	Transform	void NavEKF3_core::setAidingMode()
					{
					    resetDataSource posResetSource = resetDataSource::DEFAULT;
					    resetDataSource velResetSource = resetDataSource::DEFAULT;
					
					    // Save the previous status so we can detect when it has changed
					    PV_AidingModePrev = PV_AidingMode;
					
					    setYawSource();
						Navigation aiding-mode state machine (none/relative/absolute) driving which PNT measurements are fused. One of 13 AP_NavEKF3 fusion modules, all catalogued.	—

11	libraries/AP_NavEKF3/AP_NavEKF3_MagFusion.cpp	227-235	NavEKF3_core::SelectMagFusion()	Transform	void NavEKF3_core::SelectMagFusion()
					{
					    // clear the flag that lets other processes know that the expensive magnetometer fusion operation has been performed on that time step
					    // used for load levelling
					    magFusePerformed = false;
					
					    // Store yaw angle when moving for use as a static reference when not moving
					    if (!onGroundNotMoving) {
					        if (fabsF(prevTnb[0][2]) < fabsF(prevTnb[1][2])) {
						Magnetometer/yaw fusion scheduling feeding the heading component of the navigation solution. One of 13 AP_NavEKF3 fusion modules, all catalogued.	—

12	libraries/AP_NavEKF3/AP_NavEKF3_AirDataFusion.cpp	193-201	NavEKF3_core::SelectTasFusion()	Transform	void NavEKF3_core::SelectTasFusion()
					{
					    // Check if the magnetometer has been fused on that time step and the filter is running at faster than 200 Hz
					    // If so, don't fuse measurements on this time step to reduce frame over-runs
					    // Only allow one time slip to prevent high rate magnetometer data locking out fusion of other measurements
					    if (magFusePerformed && dtIMUavg < 0.005f && !airSpdFusionDelayed) {
					        airSpdFusionDelayed = true;
					        return;
					    } else {
						True-airspeed / synthetic-sideslip fusion (wind/velocity estimation for fixed-wing nav). One of 13 AP_NavEKF3 fusion modules, all catalogued.	—

13	libraries/AP_NavEKF3/AP_NavEKF3_OptFlowFusion.cpp	21-29	NavEKF3_core::SelectFlowFusion()	Transform	void NavEKF3_core::SelectFlowFusion()
					{
					    // Check if the magnetometer has been fused on that time step and the filter is running at faster than 200 Hz
					    // If so, don't fuse measurements on this time step to reduce frame over-runs
					    // Only allow one time slip to prevent high rate magnetometer data preventing fusion of other measurements
					    if (magFusePerformed && dtIMUavg < 0.005f && !optFlowFusionDelayed) {
					        optFlowFusionDelayed = true;
					        return;
					    } else {
						Optical-flow fusion scheduling for GPS-denied velocity/position aiding. One of 13 AP_NavEKF3 fusion modules, all catalogued.	—

<table>
<tr><th>#</th><th>File Path</th><th>Line(s)</th><th>Function/Class</th><th>Role</th><th>Code Snippet (5-10 lines)</th><th>Notes</th><th>New Service Location</th></tr>
<tr><td>14</td><td>libraries/AP_NavEKF3/AP_NavEKF3_Outputs.cpp</td><td>326-334</td><td>NavEKF3_core::getLLH()</td><td>Transform</td><td>bool NavEKF3_core::getLLH(Location &loc) const
{
    Location origin;
    if (getOriginLLH(origin)) {
        postype_t posD;
        if (getPosD_local(posD) && PV_AidingMode != AID_NONE) {
            // Altitude returned is an absolute altitude relative to the WGS-84 spherioid
            loc.set_alt_cm(origin.alt - posD*100.0, Location::AltFrame::ABSOLUTE);
            if (filterStatus.flags.horiz_pos_abs || filterStatus.flags.horiz_pos_rel) {</td><td>EKF position output: converts the filter state to a geodetic Location consumed by AP_AHRS. One of 13 AP_NavEKF3 fusion modules, all catalogued.</td><td>—</td></tr>
<tr><td>15</td><td>libraries/AP_NavEKF3/AP_NavEKF3_VehicleStatus.cpp</td><td>14-22</td><td>NavEKF3_core::calcGpsGoodToAlign()</td><td>Transform</td><td>void NavEKF3_core::calcGpsGoodToAlign(void)
{
    if (inFlight && !finalInflightYawInit && assume_zero_sideslip() && !use_compass()) {
        // this is a special case where a plane has launched without magnetometer
        // is now in the air and needs to align yaw to the GPS and start navigating as soon as possible
        gpsGoodToAlign = true;
        return;
    }</td><td>GPS-quality gate deciding whether the GPS is good enough to align/aid the navigation solution. One of 13 AP_NavEKF3 fusion modules, all catalogued.</td><td>—</td></tr>
<tr><td>16</td><td>libraries/AP_NavEKF3/AP_NavEKF3_GyroBias.cpp</td><td>6-13</td><td>NavEKF3_core::resetGyroBias()</td><td>Transform</td><td>void NavEKF3_core::resetGyroBias(void)
{
    stateStruct.gyro_bias.zero();
    zeroRows(P,10,12);
    zeroCols(P,10,12);

    P[10][10] = sq(radians(0.5f * dtIMUavg));
    P[11][11] = P[10][10];</td><td>Gyro-bias state reset within the inertial-navigation state vector. One of 13 AP_NavEKF3 fusion modules, all catalogued.</td><td>—</td></tr>
<tr><td>17</td><td>libraries/AP_NavEKF3/AP_NavEKF3_RngBcnFusion.cpp</td><td>95-103</td><td>NavEKF3_core::FuseRngBcn()</td><td>Transform</td><td>void NavEKF3_core::FuseRngBcn()
{
    // declarations
    ftype pn;
    ftype pe;
    ftype pd;
    ftype bcn_pn;
    ftype bcn_pe;
    ftype bcn_pd;</td><td>Range-beacon (non-GPS) position fusion — consumes AP_Beacon ranges (Table 1 #25). One of 13 AP_NavEKF3 fusion modules, all catalogued.</td><td>—</td></tr>
<tr><td>18</td><td>libraries/AP_NavEKF3/AP_NavEKF3_Logging.cpp</td><td>15-23</td><td>NavEKF3_core::Log_Write_XKF1()</td><td>Sink</td><td>void NavEKF3_core::Log_Write_XKF1(uint64_t time_us) const
{
    // Write first EKF packet
    Vector3f euler;
    Vector2p posNE;
    postype_t posD;
    Vector3f velNED;
    Vector3f gyroBias;
    float posDownDeriv;</td><td>Logs the fused navigation state (pos/vel/attitude) to the dataflash; reads PNT state, drives no control — a logging Sink. One of 13 AP_NavEKF3 fusion modules, all catalogued.</td><td>—</td></tr>
<tr><td>19</td><td>libraries/AP_NavEKF3/AP_NavEKF3.cpp</td><td>1846-1854</td><td>NavEKF3::getFilterStatus()</td><td>Transform</td><td>void NavEKF3::getFilterStatus(nav_filter_status &status) const
{
    if (core) {
        core[primary].getFilterStatus(status);
    } else {
        memset(&status, 0, sizeof(status));
    }
}</td><td>EKF3 frontend: exposes the per-core nav_filter_status (position/velocity validity bits) to vehicle health logic (Group 2).</td><td>—</td></tr>
<tr><td>20</td><td>libraries/AP_NavEKF2/AP_NavEKF2_core.cpp</td><td>528-536</td><td>NavEKF2_core::UpdateFilter()</td><td>Transform</td><td>void NavEKF2_core::UpdateFilter(bool predict)
{
    // Set the flag to indicate to the filter that the front-end has given permission for a new state prediction cycle to be started
    startPredictEnabled = predict;

    // don't run filter updates if states have not been initialised
    if (!statesInitialised) {
        return;
    }</td><td>[CIRCULAR] [DUPLICATED] Second-generation EKF predict/fuse step; selectable alternative to EKF3. AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference. parallel EKF implementation to AP_NavEKF3 (extraction note).</td><td>—</td></tr>
</table>

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes	New Service Location

21	libraries/AP_NavEKF2/AP_NavEKF2_PosVelFusion.cpp	560-568	NavEKF2_core::FuseVelPosNED()	Transform	void NavEKF2_core::FuseVelPosNED()
					{
					    // health is set bad until test passed
					    bool velHealth = false; // boolean true if velocity measurements have passed innovation consistency check
					    bool posHealth = false; // boolean true if position measurements have passed innovation consistency check
					    bool hgtHealth = false; // boolean true if height measurements have passed innovation consistency check
					
					    // declare variables used to check measurement errors
					    Vector3F velInnov;
						[DUPLICATED] EKF2 GPS position/velocity measurement-update; structurally mirrors EKF3 FuseVelPosNED (#8). .	—

22	libraries/AP_NavEKF/EKFGSF_yaw.cpp	30-38	EKFGSF_yaw::update()	Transform	void EKFGSF_yaw::update(const Vector3F &delAng,
					                        const Vector3F &delVel,
					                        const ftype delAngDT,
					                        const ftype delVelDT,
					                        bool runEKF,
					                        ftype TAS)
					{
					
					    // copy to class variables
						Gaussian-Sum-Filter emergency yaw estimator (shared by EKF2/EKF3) providing a GPS-velocity-derived yaw used to recover the navigation solution.	—

23	libraries/AC_WPNav/AC_WPNav.cpp	245-254	AC_WPNav::set_wp_destination_loc()	Transform	bool AC_WPNav::set_wp_destination_loc(const Location& destination)
					{
					    bool is_terrain_alt;
					    Vector3f dest_neu_cm;
					
					    // convert destination location to vector
					    if (!get_vector_NEU_cm(destination, dest_neu_cm, is_terrain_alt)) {
					        return false;
					    }
						Waypoint navigation: converts a Location waypoint to a NEU target (get_vector_NEU_cm) relative to EKF origin.	—

24	libraries/AP_Mission/AP_Mission.cpp	552-561	AP_Mission::get_next_nav_cmd()	Transform	bool AP_Mission::get_next_nav_cmd(uint16_t start_index, Mission_Command& cmd)
					{
					    // search until the end of the mission command list
					    for (uint16_t cmd_index = start_index; cmd_index < (unsigned)_cmd_total; cmd_index++) {
					        // get next command
					        if (!get_next_cmd(cmd_index, cmd, false)) {
					            // no more commands so return failure
					            return false;
					        }
					        // if found a "navigation" command then return it
						Mission-command storage reader + navigation-command sequencing: scans stored commands and returns the next NAV waypoint. NOT a PNT measurement Source — it sequences stored navigation targets (Transform).	—

25	libraries/AP_L1_Control/AP_L1_Control.cpp	226-234	AP_L1_Control::update_waypoint()	Transform	// Calculate L1 gain required for specified damping
					    float K_L1 = 4.0f * _L1_damping * _L1_damping;
					
					    // Get current position and velocity
					    if (_ahrs.get_location(_current_loc) == false) {
					        // if no GPS loc available, maintain last nav/target_bearing
					        _data_is_stale = true;
					        return;
					    }
						Fixed-wing lateral (L1) guidance consuming the AHRS navigation position via _ahrs.get_location() (returns early if unavailable). The same method also reads AP_HAL::micros() at L214 for the dt timing coupling.	AfsimL1Behavior::set_leg_ne() -> AP_L1_Control::update_waypoint(prev, next); state via AHRS shim (get_location()/groundspeed_vector()) fed by set_state_ne(); dt via set_update_dt()

26	libraries/AP_L1_Control/AP_L1_Control.cpp	79-88	AP_L1_Control::nav_roll_cd()	Transform	int32_t AP_L1_Control::nav_roll_cd(void) const
					{
					    float ret;
					/*
					formula can be obtained through equations of balanced spiral:
					liftForce * cos(roll) = gravityForce * cos(pitch);
					liftForce * sin(roll) = gravityForce * lateralAcceleration / gravityAcceleration; // as mass = gravityForce/gravityAcceleration
					see issue 24319 [https://github.com/ArduPilot/ardupilot/issues/24319]
					Multiplier 100.0f is for converting degrees to centidegrees
					Made changes to avoid zero division as proposed by Andrew Tridgell: https://github.com/ArduPilot/ardupilot/pull/24331#discussion_r1267798397
						Derives demanded roll (centi-deg) from the lateral-acceleration demand and AHRS pitch.	AfsimL1Behavior::get_roll_deg() = nav_roll_cd()/100 -> L1_GetRollDeg; get_lat_accel() -> L1_GetLatAccel

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes	New Service Location

27	libraries/AP_TECS/AP_TECS.cpp	1259-1268	AP_TECS::update_pitch_throttle()	Transform	`void AP_TECS::update_pitch_throttle(int32_t hgt_dem_cm,`
					`int32_t EAS_dem_cm,`
					`enum AP_FixedWing::FlightStage flight_stage,`
					`float distance_beyond_land_wp,`
					`int32_t ptchMinCO_cd,`
					`int16_t throttle_nudge,`
					`float hgt_afe,`
					`float load_factor,`
					`float pitch_trim_deg)`
					`{`
						Fixed-wing Total Energy Control (speed/altitude) producing pitch & throttle demands.	—

28	libraries/AC_AttitudeControl/AC_AttitudeControl.cpp	411-419	AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd()	Sink	`void AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd(float euler_roll_angle_cd, float euler_pitch_angle_cd, float euler_yaw_angle_cd, bool slew_yaw)`
					`{`
					`// Convert from centidegrees on public interface to radians`
					`const float euler_roll_angle_rad = cd_to_rad(euler_roll_angle_cd);`
					`const float euler_pitch_angle_rad = cd_to_rad(euler_pitch_angle_cd);`
					`const float euler_yaw_angle_rad = cd_to_rad(euler_yaw_angle_cd);`
					
					`input_euler_angle_roll_pitch_yaw_rad(euler_roll_angle_rad, euler_pitch_angle_rad, euler_yaw_angle_rad, slew_yaw);`
					`}`
						Attitude/rate controller consuming the navigation demand; downstream of all navigation.	—

Table 2a — Dependency Map (Layer 1 direct edges + Layer 2 transitive chains)

Table 2a: Layer 1 — direct callers / callees with typed dependency

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	AP_AHRS::get_location()	returns state.location (fused by active EKF)	ArduCopter inertia.cpp; commands.cpp; GCS_MAVLink_Copter.cpp	Function call	`bool AP_AHRS::get_location(Location &loc) const`
					`{`
					`loc = state.location;`
					`return state.location_ok;`
					`}`

2	AP_AHRS::groundspeed()	returns state.ground_speed	Rover speed control; GCS reporting	Data dependency	`// EKF has a better ground speed vector estimate`
					`const Vector2f &groundspeed_vector() const { return state.ground_speed_vec; }`
					`// return ground speed estimate in meters/second. Used by ground vehicles.`
					`float groundspeed(void) const { return state.ground_speed; }`
					`const Vector3f &get_accel_ef() const {`
					`return state.accel_ef;`
					`}`

3	AP_AHRS::have_inertial_nav()/get_origin()	active_EKF_type(); EKF origin	Copter::ekf_has_absolute_position (system.cpp); ekf_check	Function call	`// returns the inertial navigation origin in lat/lon/alt`
					`bool get_origin(Location &ret) const WARN_IF_UNUSED;`
					`bool have_inertial_nav() const;`
					`// return a ground velocity in meters/second, North/East/Down`
					`// order. Must only be called if have_inertial_nav() is true`
					`bool get_velocity_NED(Vector3f &vec) const WARN_IF_UNUSED;`

4	AP_AHRS::get_relative_position_NED_origin()	dcm/EKF2/EKF3 get_relative_position_NED_origin()	AR_PosControl::update; AP_InertialNav::update	Inheritance/Interface	`bool AP_AHRS::get_relative_position_NED_origin(Vector3p &vec) const`
					`{`
					`switch (active_EKF_type()) {`
					`#if AP_AHRS_DCM_ENABLED`
					`case EKFType::DCM:`
					`return dcm.get_relative_position_NED_origin(vec);`
					`#endif`
					`#if HAL_NAVEKF2_AVAILABLE`
					`case EKFType::TWO: {`
					`Vector2p posNE;`

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

5	AP_AHRS::get_velocity_NED()	returns state.velocity_NED	Copter::crash_check; AR_PosControl	Data dependency	bool AP_AHRS::get_velocity_NED(Vector3f &vec) const
					{
					vec = state.velocity_NED;
					return state.velocity_NED_ok;
					}

6	NavEKF3_core::UpdateFilter()	fill_scratch_variables(); Select*Fusion(); CovariancePrediction()	NavEKF3::UpdateFilter() (frontend, per core) Page	Function call	void NavEKF3_core::UpdateFilter(bool predict)
					{
					// don't run filter updates if states have not been initialise
					d
					if (!statesInitialised) {
					return;
					}
					fill_scratch_variables();
					// update sensor selection (for affinity)

7	NavEKF3_core::SelectVelPosFusi on()	readGpsData(); FuseVelPosNED()	NavEKF3_core::UpdateFilter()	Function call	void NavEKF3_core::SelectVelPosFusion()
					{
					// Check if the magnetometer has been fused on that time step
					and the filter is running at faster than 200 Hz
					// If so, don't fuse measurements on this time step to reduce
					frame over-runs
					// Only allow one time slip to prevent high rate magnetometer
					data preventing fusion of other measurements
					if (magFusePerformed && dtIMUavg < 0.005f && !posVelFusionDela
					yed) {
					posVelFusionDelayed = true;
					return;
					} else {
					posVelFusionDelayed = false;

8	NavEKF3_core::FuseVelPosNED()	Kalman gain on NED pos/vel innovations; correct stateStruct	NavEKF3_core::SelectVelPosFusion()	Function call	void NavEKF3_core::FuseVelPosNED()
					{
					// declare variables used to control access to arrays
					bool fuseData[6] {};
					uint8_t stateIndex;
					uint8_t obsIndex;
					// declare variables used by state and covariance update calcu
					lations
					Vector6 R_OBS; // Measurement variances used for fusion

9	NavEKF3_core::readGpsData()	gps.location(); gps.velocity(); gps.status() (via AP_DAL); storedGPS.push()	NavEKF3_core::SelectVelPosFusion()/U pdateFilter()	Function call	void NavEKF3_core::readGpsData()
					{
					// check for new GPS data
					const auto &gps = dal.gps();
					// limit update rate to avoid overflowing the FIFO buffer
					if (gps.last_message_time_ms(selected_gps) - lastTimeGpsReceiv
					ed_ms <= frontend->sensorIntervalMin_ms) {
					return;
					}

10	NavEKF3_core::setAidingMode()	checks gpsGoodToAlign, posTimeout; sets PV_AidingMode	NavEKF3_core::controlFilterModes()	Function call	void NavEKF3_core::setAidingMode()
					{
					resetDataSource posResetSource = resetDataSource::DEFAULT;
					resetDataSource velResetSource = resetDataSource::DEFAULT;
					// Save the previous status so we can detect when it has chang
					ed
					PV_AidingModePrev = PV_AidingMode;
					setYawSource();

11	NavEKF3_core::SelectMagFusion( )	FuseMagnetometer(); FuseEulerYaw()	NavEKF3_core::UpdateFilter() Page	Function call	void NavEKF3_core::SelectMagFusion()
					{
					// clear the flag that lets other processes know that the expe
					nsive magnetometer fusion operation has been performed on that tim
					e step
					// used for load levelling
					magFusePerformed = false;
					// Store yaw angle when moving for use as a static reference w
					hen not moving
					if (!onGroundNotMoving) {
					if (fabsF(prevTnb[0][2]) < fabsF(prevTnb[1][2])) {

<table>
<tr><th>Ref #</th><th>Component/Function</th><th>Directly Calls</th><th>Directly Called By</th><th>Dependency Type</th><th>Code Snippet</th></tr>
<tr><td>12</td><td>NavEKF3_core::SelectTasFusion()</td><td>FuseAirspeed()</td><td>NavEKF3_core::UpdateFilter()</td><td>Function call</td><td><code>void NavEKF3_core::SelectTasFusion()
{
// Check if the magnetometer has been fused on that time step and the filter is running at faster than 200 Hz
// If so, don't fuse measurements on this time step to reduce frame over-runs
// Only allow one time slip to prevent high rate magnetometer data locking out fusion of other measurements
if (magFusePerformed && dtIMUavg < 0.005f && !airSpdFusionDelayed) {
airSpdFusionDelayed = true;
return;
} else {</code></td></tr>
<tr><td>13</td><td>NavEKF3_core::SelectFlowFusion( )</td><td>EstimateTerrainOffset(); FuseOptFlow()</td><td>NavEKF3_core::UpdateFilter()</td><td>Function call</td><td><code>void NavEKF3_core::SelectFlowFusion()
{
// Check if the magnetometer has been fused on that time step and the filter is running at faster than 200 Hz
// If so, don't fuse measurements on this time step to reduce frame over-runs
// Only allow one time slip to prevent high rate magnetometer data preventing fusion of other measurements
if (magFusePerformed && dtIMUavg < 0.005f && !optFlowFusionDelayed) {
optFlowFusionDelayed = true;
return;
} else {</code></td></tr>
<tr><td>14</td><td>NavEKF3_core::getLLH()</td><td>outputDataNew / EKF_origin; builds Location</td><td>NavEKF3::getLLH() (frontend) -> AP_AHRS</td><td>Function call</td><td><code>bool NavEKF3_core::getLLH(Location &loc) const
{
Location origin;
if (getOriginLLH(origin)) {
postype_t posD;
if (getPosD_local(posD) && PV_AidingMode != AID_NONE) {
// Altitude returned is an absolute altitude relative to the WGS-84 spherioid
loc.set_alt_cm(origin.alt - posD*100.0, Location::AltFrame::ABSOLUTE);
if (filterStatus.flags.horiz_pos_abs || filterStatus.flags.horiz_pos_rel) {</code></td></tr>
<tr><td>15</td><td>NavEKF3_core::calcGpsGoodToAli gn()</td><td>reads gps metrics; sets gpsGoodToAlign</td><td>NavEKF3_core::readGpsData()/setAidingMode()</td><td>Function call</td><td><code>void NavEKF3_core::calcGpsGoodToAlign(void)
{
if (inFlight && !finalInflightYawInit && assume_zero_sideslip() && !use_compass()) {
// this is a special case where a plane has launched without magnetometer
// is now in the air and needs to align yaw to the GPS and start navigating as soon as possible
gpsGoodToAlign = true;
return;
}</code></td></tr>
<tr><td>16</td><td>NavEKF3_core::resetGyroBias()</td><td>zeroes stateStruct.gyro_bias; resets P</td><td>NavEKF3_core::InitialiseFilterBootstrap( ) Page</td><td>Function call</td><td><code>void NavEKF3_core::resetGyroBias(void)
{
stateStruct.gyro_bias.zero();
zeroRows(P,10,12);
zeroCols(P,10,12);
P[10][10] = sq(radians(0.5f * dtIMUavg));
P[11][11] = P[10][10];</code></td></tr>
<tr><td>17</td><td>NavEKF3_core::FuseRngBcn()</td><td>range innovations; correct NED position states</td><td>NavEKF3_core::SelectRngBcnFusion()</td><td>Function call</td><td><code>void NavEKF3_core::FuseRngBcn()
{
// declarations
ftype pn;
ftype pe;
ftype pd;
ftype bcn_pn;
ftype bcn_pe;
ftype bcn_pd;</code></td></tr>
</table>

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

18	NavEKF3_core::Log_Write_XKF1()	getEulerAngles(); getVelNED(); getLLH(); AP::logger().Write*	NavEKF3_core::Log_Write()	Function call	void NavEKF3_core::Log_Write_XKF1(uint64_t time_us) const
					{
					// Write first EKF packet
					Vector3f euler;
					Vector2p posNE;
					postype_t posD;
					Vector3f velNED;
					Vector3f gyroBias;
					float posDownDeriv;

19	NavEKF3::getFilterStatus()	core[].getFilterStatus()	Copter::position_ok(); ekf_check()	Function call	void NavEKF3::getFilterStatus(nav_filter_status &status) const
					{
					if (core) {
					core[primary].getFilterStatus(status);
					} else {
					memset(&status, 0, sizeof(status));
					}
					}

20	NavEKF2_core::UpdateFilter()	Select*Fusion(); CovariancePrediction()	NavEKF2::UpdateFilter() (frontend)	Function call	void NavEKF2_core::UpdateFilter(bool predict)
					{
					// Set the flag to indicate to the filter that the front-end h
					as given permission for a new state prediction cycle to be started
					startPredictEnabled = predict;
					// don't run filter updates if states have not been initialise
					d
					if (!statesInitialised) {
					return;
					}

21	NavEKF2_core::FuseVelPosNED()	Kalman gain on NED pos/vel innovations	NavEKF2_core::SelectVelPosFusion()	Function call	void NavEKF2_core::FuseVelPosNED()
					{
					// health is set bad until test passed
					bool velHealth = false; // boolean true if vel
					ocity measurements have passed innovation consistency check
					bool posHealth = false; // boolean true if pos
					ition measurements have passed innovation consistency check
					bool hgtHealth = false; // boolean true if hei
					ght measurements have passed innovation consistency check
					// declare variables used to check measurement errors
					Vector3F velInnov;

22	EKFGSF_yaw::update()	predict()/correct() per model; AHRS complementary filter	NavEKF3_core::runYawEstimatorPredic tion() Page	Function call	void EKFGSF_yaw::update(const Vector3F &delAng,
					const Vector3F &delVel,
					const ftype delAngDT,
					const ftype delVelDT,
					bool runEKF,
					ftype TAS)
					{
					// copy to class variables

23	AC_WPNav::set_wp_destination_l oc()	get_vector_NEU_cm(); set_wp_destination_NEU_cm()	ArduCopter mode_auto / mode_guided	Function call	bool AC_WPNav::set_wp_destination_loc(const Location& destination)
					{
					bool is_terrain_alt;
					Vector3f dest_neu_cm;
					// convert destination location to vector
					if (!get_vector_NEU_cm(destination, dest_neu_cm, is_terrain_al
					t)) {
					return false;
					}

24	AP_Mission::get_next_nav_cmd()	get_next_cmd(); read_cmd_from_storage()	ArduCopter/Plane mission sequencing	Function call	bool AP_Mission::get_next_nav_cmd(uint16_t start_index, Mission_Co
					mmand& cmd)
					{
					// search until the end of the mission command list
					for (uint16_t cmd_index = start_index; cmd_index < (unsigned)_
					cmd_total; cmd_index++) {
					// get next command
					if (!get_next_cmd(cmd_index, cmd, false)) {
					// no more commands so return failure
					return false;
					}
					// if found a "navigation" command then return it

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

25	AP_L1_Control::update_waypoint()	_ahrs.get_location(_current_loc)	ArduPlane navigation.cpp update loop	Function call	`// Calculate L1 gain required for specified damping`
					`float K_L1 = 4.0f * _L1_damping * _L1_damping;`
					`// Get current position and velocity`
					`if (_ahrs.get_location(_current_loc) == false) {`
					`// if no GPS loc available, maintain last nav/target_beari`
					`ng`
					`_data_is_stale = true;`
					`return;`
					`}`

26	AP_L1_Control::nav_roll_cd()	_ahrs.get_pitch_rad(); _latAccDem	Plane::calc_nav_roll() (Attitude.cpp L608)	Function call	`int32_t AP_L1_Control::nav_roll_cd(void) const`
					`{`
					`float ret;`
					`/*`
					`formula can be obtained through equations of balanced spir`
					`al:`
					`liftForce * cos(roll) = gravityForce * cos(pitch);`
					`liftForce * sin(roll) = gravityForce * lateralAcceleration`
					`/ gravityAcceleration; // as mass = gravityForce/gravityAccelerat`
					`ion`
					`see issue 24319 [https://github.com/ArduPilot/ardupilot/is`
					`sues/24319]`
					`Multiplier 100.0f is for converting degrees to centidegree`
					`s`
					`Made changes to avoid zero division as proposed by Andrew`
					`Tridgell: https://github.com/ArduPilot/ardupilot/pull/24331#discus`
					`sion_r1267798397`

27	AP_TECS::update_pitch_throttle()	_update_pitch();
		_update_throttle_with_airspeed()
			ArduPlane stabilize/auto pitch loop Page	Function call	`void AP_TECS::update_pitch_throttle(int32_t hgt_dem_cm,`
					`int32_t EAS_dem_cm,`
					`enum AP_FixedWing::FlightStage`
					`flight_stage,`
					`float distance_beyond_land_wp,`
					`int32_t ptchMinCO_cd,`
					`int16_t throttle_nudge,`
					`float hgt_afe,`
					`float load_factor,`
					`float pitch_trim_deg)`
					`{`

28	AC_AttitudeControl::input_euler_a
	ngle_roll_pitch_yaw_cd()
		input_euler_angle_roll_pitch_yaw_rad()	ArduCopter/ArduPlane flight-mode run()	Function call	`void AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd(float`
					`euler_roll_angle_cd, float euler_pitch_angle_cd, float euler_yaw_`
					`angle_cd, bool slew_yaw)`
					`{`
					`// Convert from centidegrees on public interface to radians`
					`const float euler_roll_angle_rad = cd_to_rad(euler_roll_angle_`
					`cd);`
					`const float euler_pitch_angle_rad = cd_to_rad(euler_pitch_angl`
					`e_cd);`
					`const float euler_yaw_angle_rad = cd_to_rad(euler_yaw_angle_cd`
					`);`
					`input_euler_angle_roll_pitch_yaw_rad(euler_roll_angle_rad, eul`
					`er_pitch_angle_rad, euler_yaw_angle_rad, slew_yaw);`
					`}`

Table 2a: Layer 2 — full transitive chain, final consumer & hop depth

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	AP_GPS (via AP_AHRS) — user example	`AP_GPS → AP_AHRS::get_location() → Plane::navigate() [ArduPlane/navigation.cpp:86] → ModeAuto::navigate() [ArduPlane/mode_auto.cpp:124]`	ModeAuto::navigate() [ArduPlane/mode_auto.cpp:124]	3	USER-SUPPLIED example (AAP 0.1.3/0.5.4). The literal example named 'ArduPlane::flight_mode_auto()', which does NOT exist in this snapshot; normalized to the real symbols Plane::navigate() and ModeAuto::navigate(). navigate() is gated on have_position (Table 5 #5).
2	AP_AHRS::groundspeed()	`AP_GPS velocity → NavEKF3 fusion → AP_AHRS::groundspeed() → Rover speed control → SRV_Channels`	SRV_Channels	4	Ground-vehicle speed branch.
3	AP_AHRS::have_inertial_nav()	`EKF health → AP_AHRS::have_inertial_nav() → Copter::ekf_has_absolute_position → position_ok() → set_mode gate`	mode transition (allow/deny)	4	Also observed in Group 2 (Table 5 #7).
4	AP_AHRS::get_relative_position_N ED_origin()	`EKF → AHRS NED dispatch → {AR_PosControl | AP_InertialNav} → actuators`	SRV_Channels / AP_Motors	3	Geodetic↔NED transform branch.
5	AP_AHRS::get_velocity_NED()	`EKF → AHRS → Copter::crash_check / controllers → disarm / AP_Motors`	AP_Motors / disarm	3	Velocity consumers.

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
6	NavEKF3_core::UpdateFilter()	{AP_GPS, AP_InertialSensor, AP_Baro} → NavEKF3::UpdateFilter → AP_AHRS estimate → all consumers	AP_Motors	3	[CIRCULAR] AHRS↔EKF; EKF reads AP_DAL.
7	NavEKF3 PosVel fusion (Select)	readGpsData buffer → SelectVelPosFusion → FuseVelPosNED → EKF states → AP_AHRS	AP_AHRS	4	Core fusion scheduling.
8	NavEKF3_core::FuseVelPosNED()	GPS NED innovations → FuseVelPosNED → stateStruct correction → getLLH → AP_AHRS::get_location()	AP_AHRS	4	Position/velocity measurement-update.
9	NavEKF3_core::readGpsData()	AP_GPS::location()/velocity() → readGpsData → storedGPS buffer → SelectVelPosFusion → FuseVelPosNED	EKF state correction	4	Explicit GPS→EKF ingestion hop (cf. Table 1a chains).
10	NavEKF3_core::setAidingMode()	GPS health → setAidingMode → PV_AidingMode → SelectVelPosFusion → EKF states	EKF aiding state	4	Nav aiding-mode control.
11	NavEKF3_core::SelectMagFusion( )	magnetometer → SelectMagFusion → FuseMagnetometer → yaw state → AP_AHRS heading	AP_AHRS	4	Heading component.
12	NavEKF3_core::SelectTasFusion()	airspeed → SelectTasFusion → FuseAirspeed → wind/vel states → AP_AHRS::wind_estimate	AP_AHRS	4	Wind/velocity.
13	NavEKF3_core::SelectFlowFusion( )	optical flow → SelectFlowFusion → FuseOptFlow → vel/pos states → AP_AHRS	AP_AHRS	4	GPS-denied aiding.
14	NavEKF3_core::getLLH()	EKF state → getLLH → NavEKF3::getLLH (frontend) → AP_AHRS::get_location() → nav consumers	AP_AHRS / consumers	4	EKF position output edge.
15	NavEKF3_core::calcGpsGoodToAlign()	AP_GPS metrics → calcGpsGoodToAlign → gpsGoodToAlign → setAidingMode → EKF origin set	EKF origin / aiding	4	GPS quality gate.
16	NavEKF3_core::resetGyroBias()	InitialiseFilterBootstrap → resetGyroBias → stateStruct.gyro_bias → state prediction	EKF prediction	3	Bias init.
17	NavEKF3_core::FuseRngBcn()	AP_Beacon ranges → FuseRngBcn → EKF NED position correction → AP_AHRS Page 23	AP_AHRS	3	Non-GPS position fusion.
18	NavEKF3_core::Log_Write_XKF1()	EKF states → Log_Write_XKF1 → AP_Logger → dataflash/SD log	dataflash log	3	Logging Sink branch.
19	NavEKF3::getFilterStatus()	EKF core status bits → NavEKF3::getFilterStatus → AP_AHRS → Copter::position_ok()/ekf_check()	vehicle health logic	3	Status output feeds Group 2.
20	NavEKF2_core::UpdateFilter()	{AP_GPS, INS, Baro} → NavEKF2::UpdateFilter → AP_AHRS (when EKF2 active) → consumers	AP_Motors	3	[CIRCULAR] [DUPLICATED] parallel EKF to EKF3.
21	NavEKF2_core::FuseVelPosNED()	GPS NED → NavEKF2 FuseVelPosNED → EKF2 states → AP_AHRS	AP_AHRS	3	[DUPLICATED] mirrors EKF3 #8.
22	EKFGSF_yaw::update()	AP_GPS velocity + IMU → EKFGSF_yaw::update → emergency yaw → EKF yaw reset → AP_AHRS	AP_AHRS	4	Yaw-recovery branch.
23	AC_WPNav	AP_AHRS pos → AC_WPNav target → AC_PosControl → AC_AttitudeControl → AP_Motors	AP_Motors	4	Multirotor waypoint path.
24	AP_Mission	mission storage → get_next_nav_cmd → mode_auto → AC_WPNav → controllers	AP_Motors	4	Mission sequencing path.
25	AP_L1_Control::update_waypoint()	AP_AHRS pos → L1 guidance → nav_roll_cd() → Plane::calc_nav_roll → SRV_Channels	SRV_Channels	4	Fixed-wing lateral path.
26	AP_L1_Control::nav_roll_cd()	L1 latAccDem → nav_roll_cd() → Plane::calc_nav_roll → SRV_Channels	SRV_Channels	3	Roll-demand branch.
27	AP_TECS	airspeed/alt demand → TECS update_pitch_throttle → pitch/throttle → SRV_Channels	SRV_Channels	3	Energy/altitude branch.
28	AC_AttitudeControl (Sink)	nav demand → input_euler_* → attitude_controller_run_quat → rate_controller_run → AP_Motors Page 24	AP_Motors	4	Inbound chain to attitude Sink.

Table 3 — Core Timing

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes

1	libraries/AP_HAL/system.h	15-20	AP_HAL::micros()/millis()/micros64()/millis64()	Source	`uint16_t micros16();`
					`uint32_t micros();`
					`uint32_t millis();`
					`uint16_t millis16();`
					`uint64_t micros64();`
					`uint64_t millis64();`
						Monotonic time backbone (user-cited micros()/millis()). Free functions in the AP_HAL namespace backed by the board RTOS timer; the time primitive every PNT subsystem stamps against.

2	libraries/AP_RTC/AP_RTC.h	32-40	AP_RTC::get_utc_usec()	Source	`/*`
					`  get clock in UTC microseconds. Returns false if it is not available.`
					` */`
					`bool get_utc_usec(uint64_t &usec) const;`
					
					`// set the system time. If the time has already been set by`
					`// something better (according to source_type), this set will be`
					`// ignored.`
					`void set_utc_usec(uint64_t time_utc_usec, source_type type);`
						UTC wall-clock time in microseconds (returns false if unavailable). The absolute-time API consumed for logging and GPS-time conversion.

#	File Path	Line(s)	Function/Class	Role	Code Snippet (5-10 lines)	Notes

3	libraries/AP_RTC/AP_RTC.cpp	56-65	AP_RTC::set_utc_usec()	Transform	// only allow time to be moved forward from the same sourcetype
					// while the vehicle is disarmed:
					if (hal.util->get_soft_armed()) {
					if (type >= rtc_source_type) {
					// e.g. system-time message when we've been set by the GPS
					return;
					}
					} else {
					// vehicle is disarmed; accept (e.g.) GPS time source updates
					if (type > rtc_source_type) {
						Time-SOURCE ARBITRATION: accepts a new UTC time only if its source_type out-ranks the current rtc_source_type (GPS > GCS > HW). Selects the best clock source.

4	libraries/AP_RTC/JitterCorrection.cpp	50-58	JitterCorrection::correct_offboard_timestamp_usec()	Transform	uint64_t JitterCorrection::correct_offboard_timestamp_usec(uint64_t offboard_usec, uint64_t local_usec)
					{
					int64_t diff_us = int64_t(local_usec) - int64_t(offboard_usec);
					
					if (!initialised ||
					diff_us < link_offset_usec) {
					// this message arrived from the remote system with a
					// timestamp that would imply the message was from the
					// future. We know that isn't possible, so we adjust down the
						Time synchronization: maps an offboard (remote-system) timestamp onto the local clock by tracking the minimum observed link offset — rejects 'from the future' samples.

5	libraries/AP_Scheduler/AP_Scheduler.cpp	354-363	AP_Scheduler::loop()	Transform	hal.util->persistent_data.scheduler_task = -1;
					
					_loop_sample_time_us = AP_HAL::micros64();
					const uint32_t sample_time_us = uint32_t(_loop_sample_time_us);
					
					if (_loop_timer_start_us == 0) {
					_loop_timer_start_us = sample_time_us;
					_last_loop_time_s = get_loop_period_s();
					} else {
					_last_loop_time_s = (sample_time_us - _loop_timer_start_us) * 1.0e-6;
						Main-loop timing core: stamps each loop with AP_HAL::micros64() and derives the measured loop time. Drives the cadence of every periodic PNT task (EKF predict/fuse, AHRS, control).

6	libraries/AP_Scheduler/AP_Scheduler.cpp	168-176	AP_Scheduler::tick()	Source	void AP_Scheduler::tick(void)
					{
					_tick_counter++;
					_tick_counter32++;
					}
					
					#if CONFIG_HAL_BOARD == HAL_BOARD_SITL
					/*
					fill stack with NaN so we can catch use of uninitialised stack
						Increments the loop tick counters (_tick_counter / _tick_counter32) that index task scheduling intervals.

7	libraries/AP_Scheduler/AP_Scheduler.h	149-155	AP_Scheduler::get_loop_period_us()	Transform	// get the time-allowed-per-loop in microseconds
					uint32_t get_loop_period_us() {
					if (_loop_period_us == 0) {
					_loop_period_us = 1000000UL / _loop_rate_hz;
					}
					return _loop_period_us;
					}
						Loop-rate→period derivation: _loop_period_us = 1000000UL / _loop_rate_hz. Couples the configured loop rate to the per-loop time budget and to controller/EKF dt.

8	libraries/AP_GPS/AP_GPS.h	409-418	AP_GPS::time_week_ms()/time_week()	Source	// GPS time of week in milliseconds
					uint32_t time_week_ms(uint8_t instance) const {
					return state[instance].time_week_ms;
					}
					uint32_t time_week_ms() const {
					return time_week_ms(primary_instance);
					}
					
					// GPS week
					uint16_t time_week(uint8_t instance) const {
						[MULTI-CATEGORY] GPS time-of-week (ms-of-week + week number) — the TIMING facet of the GPS_State that also carries Positioning (Table 1 #4/#17). Emitted separately here per the multi-category rule.

9	libraries/AP_HAL/Scheduler.h	13-21	AP_HAL::Scheduler::delay()/delay_microseconds()	Source	Scheduler() {}
					virtual void init() = 0;
					virtual void delay(uint16_t ms) = 0;
					
					/*
					delay for the given number of microseconds. This needs to be as
					accurate as possible - preferably within 100 microseconds.
					*/
					virtual void delay_microseconds(uint16_t us) = 0;
						Abstract HAL timing-service interface (pure virtual; implemented by the board backend — ChibiOS / Linux / SITL). Supplies the time-blocking primitives delay() / delay_microseconds() and the periodic-task registration register_timer_process() / register_io_process() (lines 60-64) on which the main loop is built. Distinct from the AP_Scheduler library (#5-#7) and the AP_HAL::micros()/millis() clock primitives in system.h (#1).

Table 3a — Dependency Map (Layer 1 direct edges + Layer 2 transitive chains)

Table 3a: Layer 1 — direct callers / callees with typed dependency

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	AP_HAL::micros() / millis()	board RTOS/HAL timer backend	EKF, AP_Scheduler, AP_GPS, all failsafes (ubiquitous)	Function call	`uint16_t micros16();`
					`uint32_t micros();`
					`uint32_t millis();`
					`uint16_t millis16();`
					`uint64_t micros64();`
					`uint64_t millis64();`

2	AP_RTC::get_utc_usec() — via AP::rtc()	AP::rtc().get_utc_usec(rtc)	AP_Scheduler::Log_Write_Performance ; AP_Logger; AP_Stats; AP_NMEA_Output	Shared global/singleton	`void AP_Scheduler::Log_Write_Performance()`
					`{`
					`uint64_t rtc = 0;`
					`#if AP_RTC_ENABLED`
					`UNUSED_RESULT(AP::rtc().get_utc_usec(rtc));`
					`#endif`
					`const AP_HAL::Util::PersistentData &pd = hal.util->persistent_`
					`data;`
					`struct log_Performance pkt = {`

3	AP_RTC::set_utc_usec() (source arbitration)	reads hal.util->get_soft_armed(); compares & writes rtc_source_type	AP_GPS (GPS time); GCS SYSTEM_TIME; AP_BoardConfig (HW RTC) Page	Data dependency	`// only allow time to be moved forward from the same sourcetyp`
					`e`
					`// while the vehicle is disarmed:`
					`if (hal.util->get_soft_armed()) {`
					`if (type >= rtc_source_type) {`
					`// e.g. system-time message when we've been set by the`
					`GPS`
					`return;`
					`}`
					`} else {`
					`// vehicle is disarmed; accept (e.g.) GPS time source upda`
					`tes`
					`if (type > rtc_source_type) {`

4	JitterCorrection::correct_offboard_timestamp_usec()	computes diff_us; updates link_offset_usec	AP_RTC; DroneCAN/MAVLink timestamp consumers	Function call	`uint64_t JitterCorrection::correct_offboard_timestamp_usec(uint64_`
					`t offboard_usec, uint64_t local_usec)`
					`{`
					`int64_t diff_us = int64_t(local_usec) - int64_t(offboard_usec)`
					`;`
					`if (!initialised ||`
					`diff_us < link_offset_usec) {`
					`// this message arrived from the remote system with a`
					`// timestamp that would imply the message was from the`
					`// future. We know that isn't possible, so we adjust down`
					`the`

5	AP_Scheduler::loop()	AP_HAL::micros64(); get_loop_period_s(); dispatches run-table tasks	AP_Vehicle::loop() via scheduler.loop() (AP_Vehicle.cpp:545)	Function call	`hal.util->persistent_data.scheduler_task = -1;`
					`_loop_sample_time_us = AP_HAL::micros64();`
					`const uint32_t sample_time_us = uint32_t(_loop_sample_time_us)`
					`;`
					`if (_loop_timer_start_us == 0) {`
					`_loop_timer_start_us = sample_time_us;`
					`_last_loop_time_s = get_loop_period_s();`
					`} else {`
					`_last_loop_time_s = (sample_time_us - _loop_timer_start_us`
					`) * 1.0e-6;`

6	AP_Scheduler::tick()	increments _tick_counter / _tick_counter32	AP_Scheduler::loop(); fast-loop	Function call	`void AP_Scheduler::tick(void)`
					`{`
					`_tick_counter++;`
					`_tick_counter32++;`
					`}`
					`#if CONFIG_HAL_BOARD == HAL_BOARD_SITL`
					`/*`
					`fill stack with NaN so we can catch use of uninitialised stack`

7	AP_Scheduler::get_loop_period_u s()	derives _loop_period_us from _loop_rate_hz	task budgeting; controller dt (G_Dt)	Data dependency	`// get the time-allowed-per-loop in microseconds`
					`uint32_t get_loop_period_us() {`
					`if (_loop_period_us == 0) {`
					`_loop_period_us = 1000000UL / _loop_rate_hz;`
					`}`
					`return _loop_period_us;`
					`}`

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

8	AP_GPS::time_week_ms() / time_week()	returns state[instance].time_week_ms / time_week	NavEKF3 readGpsData (GPS time); time_epoch conversion; logging	Data dependency	`// GPS time of week in milliseconds`
					`uint32_t time_week_ms(uint8_t instance) const {`
					`return state[instance].time_week_ms;`
					`}`
					`uint32_t time_week_ms() const {`
					`return time_week_ms(primary_instance);`
					`}`
					`// GPS week`
					`uint16_t time_week(uint8_t instance) const {`

9	AP_HAL::Scheduler (abstract HAL timing / scheduling interface)	pure virtual — implemented by the board backend (ChibiOS / Linux / SITL HAL Scheduler)	AP_Scheduler via hal.scheduler->delay_microseconds (AP_Scheduler.cpp:373); register_io_process; AP_Vehicle init; all subsystems	Inheritance/Interface	`// register a high priority timer task`
					`virtual void register_timer_process(AP_HAL::MemberProc) =`
					`0;`
					`// register a low priority IO task`
					`virtual void register_io_process(AP_HAL::MemberProc) = 0;`

Table 3a: Layer 2 — full transitive chain, final consumer & hop depth

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	RTOS/board hardware timer	`board timer → AP_HAL::micros()/millis() → AP_Scheduler / EKF / AP_GPS timestamps → every periodic loop Page 27`	all periodic PNT subsystems	3	Monotonic-time backbone.
2	GPS/GCS UTC time source	`time source → AP_RTC → AP::rtc().get_utc_usec() → AP_Logger / AP_Stats → log timestamps`	dataflash log timestamps	4	Singleton consumption edge (AP::rtc()).
3	AP_GPS time / GCS SYSTEM_TIME	`time source → set_utc_usec source arbitration → rtc_source_type → system UTC clock`	RTC system clock	3	Best-source selection.
4	Offboard (remote) timestamp	`MAVLink/DroneCAN timestamp → correct_offboard_timestamp_usec → corrected local time → EKF / sensor fusion`	fused-sensor timing	3	Time-sync transform.
5	AP_HAL::micros64() (loop clock)	`AP_HAL::micros64() → AP_Scheduler::loop() → run-table tasks (EKF/AHRS/controllers) → AP_Motors`	AP_Motors (all tasks)	3	Loop drives every PNT task.
6	AP_Scheduler::loop()	`loop() → tick() counters → task-table interval check → periodic task dispatch`	periodic task dispatch	3	Cadence counter.
7	SCHED_LOOP_RATE parameter	`_loop_rate_hz → get_loop_period_us (1e6/rate) → task budget + G_Dt → AC_AttitudeControl / EKF dt`	controller / EKF dt	3	Rate→period coupling.
8	GPS receiver (time-of-week)	`GPS receiver → GPS_State.time_week → AP_GPS::time_week() → NavEKF3 readGpsData / time_epoch_usec → UTC/log`	EKF GPS-time / UTC conversion	4	[MULTI-CATEGORY] timing facet of GPS.
9	board RTOS scheduler (HAL backend)	`board HAL Scheduler backend → AP_HAL::Scheduler interface (delay / register_timer_process) → AP_Scheduler::loop → periodic PNT tasks (EKF / AHRS / control) Page 28`	periodic PNT task execution	3	HAL timing interface beneath the AP_Scheduler library (#5-#7); the board backend implements the pure-virtual interface.

GROUP 2 — INDIRECT / RELATIONAL PNT REFERENCES

Table 4 — Indirect Positioning

<table>
<tr><th>#</th><th>File Path</th><th>Line(s)</th><th>Function/Class</th><th>Trigger Condition</th><th>PNT State Observed</th><th>Behavior Change Description</th><th>Code Snippet (5-10 lines)</th><th>Notes</th></tr>
<tr><td>1</td><td>ArduCopter/system.cpp</td><td>226-234</td><td>Copter::position_ok()</td><td>Evaluated on position-mode entry and by position-dependent failsafes</td><td>failsafe.ekf flag; ekf_has_ absolute_position() / ekf_ has_relative_position() (EKF position validity)</td><td>Returns false when EKF failsafe is set or no EKF position estimate exists, causing callers to deny position-requiring behavior.</td><td>bool Copter::position_ok() const
{
    // return false if ekf failsafe has triggered
    if (failsafe.ekf) {
        return false;
    }

    // check ekf position estimate
    return (ekf_has_absolute_position() || ekf_has_relative_position());</td><td>INDIRECT/RELATIONAL (positioning). Pure observer of EKF position health — reads no raw PNT, but gates downstream behavior.</td></tr>
<tr><td>2</td><td>ArduCopter/mode.cpp</td><td>303-310</td><td>Copter::set_mode()—re quires_GPSgate</td><td>Pilot/GCS/auto requests a flight-mode change</td><td>new_flightmode->require s_GPS() && copter.position_ok()</td><td>Rejects the mode change with “requires position” when the target mode needs GPS and position_ok() is false.</td><td>   if (!ignore_checks &&
        new_flightmode->requires_GPS() &&
        !copter.position_ok()) {
        mode_change_failed(new_flightmode, "requires position");
        return false;
    }</td><td>INDIRECT/RELATIONAL (positioning). Central position-gate for ALL position-requiring Copter modes (per-mode requires_GPS() enumerated in the coverage register).</td></tr>
<tr><td>3</td><td>ArduCopter/fence.cpp</td><td>20-28</td><td>Copter::fence_checks_a sync()</td><td>25 Hz fence rate-gate elapsed and no breach latched awaiting the main loop</td><td>fence.get_breaches() (current breach bitmask, L21); fence.check() (re-evaluates vehicle position vs fences, L25)</td><td>Latches new_breaches bitmask for the main loop and logs a breach-cleared event when the vehicle returns inside the fence.</td><td>   fence_breaches.last_check_ms = now;
    const uint8_t orig_breaches = fence.get_breaches();
    bool is_landing_or_landed = flightmode->is_landing() || ap.land_complete || !motors->armed();

    // check for new breaches; new_breaches is bitmask of fence types breached
    fence_breaches.new_breaches = fence.check(is_landing_or_landed);

    if (!fence_breaches.new_breaches && orig_breaches && fence.get_breaches() == 0) {
        if (!copter.ap.land_complete) {</td><td>[DUPLICATED] [FIX F1] Cited window centers on get_breaches() (L21) and check() (L25) — the actual geofence observation — not the L8-17 async/timing entry. mirrors Rover::fence_checks_async (#9). Timing rate-gate documented in Table 6.</td></tr>
<tr><td>4</td><td>libraries/AC_Fence/AC_F ence.cpp</td><td>677-685</td><td>AC_Fence::check_fence_ polygon()</td><td>Polygon (inclusion/exclusion) fence enabled</td><td>AP::ahrs().get_location(lo c) — current vehicle Location tested against the polygon</td><td>record_breach(POLYGON) when _poly_loader.breached(loc) is true sets the polygon breach → bit. Page 29</td><td>   const bool was_breached = _breached_fences & AC_FENCE_TYPE_POLYGON;

    Location loc;
    const bool have_location = AP::ahrs().get_location(loc);

    if (have_location && _poly_loader.breached(loc, _polygon_breach_distance)) {
        if (!was_breached) {
            record_breach(AC_FENCE_TYPE_POLYGON);
            return true;</td><td>[FIX F2] Real position-observing path (was mis-cited to L753-762 mask init). Snippet centers on the AP::ahrs().get_location() read (L680).</td></tr>
<tr><td>5</td><td>libraries/AC_Fence/AC_F ence.cpp</td><td>707-715</td><td>AC_Fence::check_fence_ circle()</td><td>Circle fence enabled</td><td>AP::ahrs().get_relative_p osition_NE_home(home) — NE distance from home (L713)</td><td>Records a circle breach when the home distance exceeds the configured radius.</td><td>   if (!(get_enabled_fences() & AC_FENCE_TYPE_CIRCLE)) {
        // not enabled; no breach
        return false;
    }

    Vector2f home;
    if (AP::ahrs().get_relative_position_NE_home(home)) {
        // we (may) remain breached if we can't update home
        _home_distance = home.length();</td><td>[FIX F2] Real position-observing path; snippet centers on the NE-home position read.</td></tr>
</table>

#	File Path	Line(s)	Function/Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

6	libraries/AC_Fence/AC_Fence.cpp	540-548	AC_Fence::check_fence_alt_max()	Maximum-altitude fence enabled	AP::ahrs().get_relative_position_D_home(alt) — altitude relative to home (L546)	Records an altitude breach when alt exceeds _alt_max.	`if (!(get_enabled_fences() & AC_FENCE_TYPE_ALT_MAX)) {`
							`// not enabled; no breach`
							`return false;`
							`}`
							
							`float alt;`
							`AP::ahrs().get_relative_position_D_home(alt);`
							`const float _curr_alt = -alt; // translate Down to Up`
								[FIX F2] Real altitude-observing path; snippet centers on the D-home altitude read.

7	libraries/AC_Fence/AC_Fence.cpp	837-845	AC_Fence::check()	Periodic fence check invoked from the vehicle fence_checks_async()	(aggregator) results of check_fence_alt_max() / check_fence_circle() / check_fence_polygon() position checks	Returns the bitmask of breached fence types that drives the vehicle fence failsafe action.	`if (!(disabled_fences & AC_FENCE_TYPE_CIRCLE) && check_fence_circle()) {`
							`ret |= AC_FENCE_TYPE_CIRCLE;`
							`}`
							
							`// polygon fence check`
							`if (!(disabled_fences & AC_FENCE_TYPE_POLYGON) && check_fence_polygon()) {`
							`ret |= AC_FENCE_TYPE_POLYGON;`
							`}`
								[FIX F2] Orchestrator dispatch (L837 circle, L842 polygon). The position reads themselves are in the dispatched checks (#4-#6).

8	ArduCopter/mode_guided_nogps.cpp	9-15	ModeGuidedNoGPS::init()	GUIDED_NOGPS entry (companion-computer angle control without GPS)	[ABSENT — by design] no position state observed; requires_GPS() returns false (ArduCopter/mode.h)	Starts angle control only and remains available when position_ok() is false — the GPS-denied fallback path.	`// initialise guided_nogps controller`
							`bool ModeGuidedNoGPS::init(bool ignore_checks)`
							`{`
							`// start in angle control mode`
							`ModeGuided::angle_control_start();`
							`return true;`
							`}`
								INDIRECT/RELATIONAL (positioning). Contrast row: intentionally PNT-position-INDEPENDENT mode (documents an explicit absence of positioning dependency).

9	Rover/fence.cpp	17-25	Rover::fence_checks_async()	Rover fence rate-gate elapsed and no breach latched	fence.get_breaches() (L19); fence.check() (vehicle position vs fence, L21)	Latches new_breaches for the main loop; clears on return inside the fence. Page 30	`fence_breaches.last_check_ms = now;`
							`const uint8_t orig_breaches = fence.get_breaches();`
							`// check for new breaches; new_breaches is bitmask of fence types breached`
							`fence_breaches.new_breaches = fence.check();`
							
							`if (!fence_breaches.new_breaches && orig_breaches && fence.get_breaches() == 0)`
							`{`
							`// record clearing of breach`
							`LOGGER_WRITE_ERROR(LogErrorSubsystem::FAILSAFE_FENCE, LogErrorCode::ERROR_RESOLVED);`
								[DUPLICATED] structurally identical to ArduCopter::fence_checks_async (#3) — copy-paste across vehicles (extraction risk).

10	ArduPlane/takeoff.cpp	58-62	Plane::auto_takeoff_check()	Auto/TAKEOFF launch readiness re-evaluated each loop from Plane::set_servos() (ArduPlane/servos.cpp:125)	gps.status() < AP_GPS::GPS_OK_FIX_3D (GPS fix quality; gps.status() = AP_GPS::status())	Returns false — auto-takeoff is blocked (throttle stays suppressed) until a 3D GPS fix is acquired.	`// Check for bad GPS`
							`if (gps.status() < AP_GPS::GPS_OK_FIX_3D) {`
							`// no auto takeoff without GPS lock`
							`return false;`
							`}`
								[DUPLICATED] INDIRECT/RELATIONAL (positioning). User-example PNT-State-Observed accessor gps.status() (= AP_GPS::status(), libraries/AP_GPS/AP_GPS.h:293). the same gps.status() < GPS_OK_FIX_3D fix-gate recurs in ArduPlane/Plane.cpp (L420/485) and ArduPlane/is_flying.cpp (extraction risk).

Table 4a — Dependency Map (Layer 1 direct edges + Layer 2 transitive chains)

Table 4a: Layer 1 — direct callers / callees with typed dependency

<table>
<tr><th>Ref #</th><th>Component/Function</th><th>Directly Calls</th><th>Directly Called By</th><th>Dependency Type</th><th>Code Snippet</th></tr>
<tr><td>1</td><td>Copter::position_ok()</td><td>ekf_has_absolute_position(); ekf_has_relative_position(); reads failsafe.ekf</td><td>Copter::set_mode (mode.cpp:306); position-dependent failsafes</td><td>Function call</td><td>bool Copter::position_ok() const
{
// return false if ekf failsafe has triggered
if (failsafe.ekf) {
return false;
}
// check ekf position estimate
return (ekf_has_absolute_position() || ekf_has_relative_positi
on());</td></tr>
<tr><td>2</td><td>Copter::set_mode() requires_GPS gate</td><td>new_flightmode->requires_GPS(); copter.position_ok()</td><td>GCS/RC/auto mode-change entry points</td><td>Function call</td><td>if (!ignore_checks &&
new_flightmode->requires_GPS() &&
!copter.position_ok()) {
mode_change_failed(new_flightmode, "requires position");
return false;
}</td></tr>
<tr><td>3</td><td>Copter::fence_checks_async()</td><td>fence.get_breaches(); fence.check()</td><td>Copter IO thread (1 kHz async, 25 Hz gated)</td><td>Function call</td><td>fence_breaches.last_check_ms = now;
const uint8_t orig_breaches = fence.get_breaches();
bool is_landing_or_landed = flightmode->is_landing() || ap.lan
d_complete || !motors->armed();
// check for new breaches; new_breaches is bitmask of fence ty
pes breached
fence_breaches.new_breaches = fence.check(is_landing_or_landed
);
if (!fence_breaches.new_breaches && orig_breaches && fence.get
_breaches() == 0) {
if (!copter.ap.land_complete) {</td></tr>
<tr><td>4</td><td>AC_Fence::check_fence_polygon()</td><td>AP::ahrs().get_location(); _poly_loader.breached()</td><td>AC_Fence::check() (L842) Page</td><td>Shared global/singleton</td><td>const bool was_breached = _breached_fences & AC_FENCE_TYPE_POL
YGON;
Location loc;
const bool have_location = AP::ahrs().get_location(loc);
if (have_location && _poly_loader.breached(loc, _polygon_breac
h_distance)) {
if (!was_breached) {
record_breach(AC_FENCE_TYPE_POLYGON);
return true;</td></tr>
<tr><td>5</td><td>AC_Fence::check_fence_circle()</td><td>AP::ahrs().get_relative_position_NE_ho me()</td><td>AC_Fence::check() (L837)</td><td>Shared global/singleton</td><td>if (!(get_enabled_fences() & AC_FENCE_TYPE_CIRCLE)) {
// not enabled; no breach
return false;
}
Vector2f home;
if (AP::ahrs().get_relative_position_NE_home(home)) {
// we (may) remain breached if we can't update home
_home_distance = home.length();</td></tr>
<tr><td>6</td><td>AC_Fence::check_fence_alt_max()</td><td>AP::ahrs().get_relative_position_D_hom e()</td><td>AC_Fence::check() (L821)</td><td>Shared global/singleton</td><td>if (!(get_enabled_fences() & AC_FENCE_TYPE_ALT_MAX)) {
// not enabled; no breach
return false;
}
float alt;
AP::ahrs().get_relative_position_D_home(alt);
const float _curr_alt = -alt; // translate Down to Up</td></tr>
<tr><td>7</td><td>AC_Fence::check()</td><td>check_fence_alt_max(); check_fence_circle(); check_fence_polygon()</td><td>Copter::fence_checks_async(); Rover::fence_checks_async()</td><td>Function call</td><td>if (!(disabled_fences & AC_FENCE_TYPE_CIRCLE) && check_fence_c
ircle()) {
ret |= AC_FENCE_TYPE_CIRCLE;
}
// polygon fence check
if (!(disabled_fences & AC_FENCE_TYPE_POLYGON) && check_fence_
polygon()) {
ret |= AC_FENCE_TYPE_POLYGON;
}</td></tr>
<tr><td>8</td><td>ModeGuidedNoGPS::init()</td><td>ModeGuided::angle_control_start()</td><td>Copter::set_mode()</td><td>Inheritance/Interface</td><td>// initialise guided_nogps controller
bool ModeGuidedNoGPS::init(bool ignore_checks)
{
// start in angle control mode
ModeGuided::angle_control_start();
return true;
}</td></tr>
</table>

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

9	Rover::fence_checks_async()	fence.get_breaches(); fence.check()	Rover IO thread	Function call	`fence_breaches.last_check_ms = now;`
					`const uint8_t orig_breaches = fence.get_breaches();`
					`// check for new breaches; new_breaches is bitmask of fence ty pes breached`
					`fence_breaches.new_breaches = fence.check();`
					`if (!fence_breaches.new_breaches && orig_breaches && fence.get _breaches() == 0) {`
					`// record clearing of breach`
					`LOGGER_WRITE_ERROR(LogErrorSubsystem::FAILSAFE_FENCE, LogE rrorCode::ERROR_RESOLVED);`

10	Plane::auto_takeoff_check()	AP_GPS::status() (gps.status()); compares vs AP_GPS::GPS_OK_FIX_3D	Plane::set_servos() (ArduPlane/servos.cpp:125)	Function call	`// Check for bad GPS`
					`if (gps.status() < AP_GPS::GPS_OK_FIX_3D) {`
					`// no auto takeoff without GPS lock`
					`return false;`
					`}`

Table 4a: Layer 2 — full transitive chain, final consumer & hop depth

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	EKF position health (via AP_AHRS)	`AP_GPS → NavEKF3 → AP_AHRS::have_inertial_nav() → Copter::ekf_has_absolute_position() → Copter::position_ok() → set_mode gate`	flight-mode entry allow/deny	5	Core positioning-health gate.
2	Copter::position_ok()	`position_ok() → set_mode requires_GPS gate → mode_change_failed() → mode unchanged`	rejected mode change	3	Gate decision path.
3	AP_AHRS vehicle position (via AC_Fence)	`AP::ahrs() position → AC_Fence::check (polygon/circle/alt) → get_breaches bitmask → fence_checks_async latch → Copter::fence_check() action Page 32`	Copter fence failsafe (RTL/LAND)	4	[DUPLICATED] mirrors Rover chain #9.
4	AP_AHRS::get_location()	`AP_AHRS::get_location() → check_fence_polygon → _poly_loader.breached() → record_breach() → _breached_fences`	polygon breach bit	4	Polygon position test.
5	AP_AHRS NE-home position	`AP::ahrs().get_relative_position_NE_home() → check_fence_circle → distance>radius → record_breach()`	circle breach bit	3	Circle position test.
6	AP_AHRS D-home altitude	`AP::ahrs().get_relative_position_D_home() → check_fence_alt_max → alt>_alt_max → record_breach()`	altitude breach bit	3	Altitude position test.
7	AC_Fence position checks	`position checks → AC_Fence::check() aggregate → breach bitmask → vehicle fence_check() → failsafe action`	vehicle fence failsafe	4	Aggregator path.
8	(no PNT — GPS-independent)	`ModeGuidedNoGPS::init() → angle_control_start() → AC_AttitudeControl → AP_Motors`	AP_Motors	3	[ABSENT] no positioning hop — fallback by design.
9	AP_AHRS vehicle position (via AC_Fence)	`AP::ahrs() position → AC_Fence::check → get_breaches → Rover::fence_checks_async latch → Rover::fence_check() action`	Rover fence failsafe	4	[DUPLICATED] structurally identical to Copter chain #3.
10	AP_GPS::status() (GPS fix-status accessor)	`AP_GPS::status() → Plane::auto_takeoff_check() [ArduPlane/takeoff.cpp:59] → Plane::set_servos() [ArduPlane/servos.cpp:125] Page 33`	Plane::set_servos() (throttle suppression / launch gate)	2	User-example gps.status() observation gating auto-takeoff.

Table 5 — Indirect Navigation

#	File Path	Line(s)	Function/Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

1	ArduCopter/ekf_check.cpp	117-125	Copter::ekf_over_threshold()	10 Hz EKF health check while the EKF has an origin	ahrs.get_variances() → velocity / position / height / magnetometer variances vs g.fs_ekf_thresh	Counts bad iterations; after EKF_CHECK_ITERATIONS_MAX (≈10) trips the EKF failsafe (LAND / AltHold).	`float position_var, vel_var, height_var, tas_variance;`
							`Vector3f mag_variance;`
							`variances_valid = ahrs.get_variances(vel_var, position_var, height_var, mag_variance, tas_variance);`
							
							`if (!variances_valid) {`
							`return false;`
							`}`
							
							`uint32_t now_us = AP_HAL::micros();`
								[DUPLICATED] INDIRECT/RELATIONAL (navigation). EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk).

#	File Path	Line(s)	Function/Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

2	ArduPlane/ekf_check.cpp	56-65	Plane::ekf_check()	10 Hz EKF health check (fixed-wing)	ekf_over_threshold() (compass & velocity variance vs fs_ekf_thresh)	Increments fail_count; two iterations before failsafe asks the EKF to reset yaw (ahrs.request_yaw_reset()); trips EKF failsafe at MAX.	`// compare compass and velocity variance vs threshold`
							`if (ekf_over_threshold()) {`
							`// if compass is not yet flagged as bad`
							`if (!ekf_check_state.bad_variance) {`
							`// increase counter`
							`ekf_check_state.fail_count++;`
							`if (ekf_check_state.fail_count == (EKF_CHECK_ITERATIONS_MAX-2)) {`
							`// we are two iterations away from declaring an EKF failsafe, ask the EKF if we can reset`
							`// yaw to resolve the issue`
							`ahrs.request_yaw_reset();`
								[DUPLICATED] INDIRECT/RELATIONAL (navigation). EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk).

3	Rover/ekf_check.cpp	93-102	Rover::ekf_over_threshold()	10 Hz EKF health check (ground vehicle)	ahrs.get_variances() → compass / velocity / position variance; ≥2 over threshold	Sets bad_variance Rover EKF → failsafe action.	`// use EKF to get variance`
							`float position_variance, vel_variance, height_variance, tas_variance;`
							`Vector3f mag_variance;`
							`ahrs.get_variances(vel_variance, position_variance, height_variance, mag_variance, tas_variance);`
							
							`// return true if two of compass, velocity and position variances are over the threshold`
							`uint8_t over_thresh_count = 0;`
							`if (mag_variance.length() >= g.fs_ekf_thresh) {`
							`over_thresh_count++;`
								[DUPLICATED] INDIRECT/RELATIONAL (navigation). EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk).

4	ArduSub/failsafe.cpp	112-121	Sub::failsafe_ekf_check()	Periodic EKF health check (submarine)	ahrs.get_variances() → compass_variance & vel_variance vs g.fs_ekf_thresh	Clears/sets failsafe.ekf and AP_Notify ekf_bad based on variance vs threshold; GCS warns “EKF bad”. Page 34	`Vector3f magVar;`
							`float compass_variance;`
							`float vel_variance;`
							`ahrs.get_variances(vel_variance, posVar, hgtVar, magVar, tasVar);`
							`compass_variance = magVar.length();`
							
							`if (compass_variance < g.fs_ekf_thresh && vel_variance < g.fs_ekf_thresh) {`
							`last_ekf_good_ms = AP_HAL::millis();`
							`failsafe.ekf = false;`
							`AP_Notify::flags.ekf_bad = false;`
								[DUPLICATED] INDIRECT/RELATIONAL (navigation). [FIX F13 dep-type] EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk).

5	ArduCopter/crash_check.cpp	65-74	Copter::crash_check()—attitudetest	Disarm/crash monitor running while armed and not landed	ahrs.cos_roll() / ahrs.cos_pitch() (lean angle); attitude_control->get_att_error_angle_deg( )	Resets crash_counter (declares not-crashing) when lean angle ≤ 15° or attitude error 30°. ≤	`// check for lean angle over 15 degrees`
							`const float lean_angle_deg = degrees(acosf(ahrs.cos_roll()*ahrs.cos_pitch()));`
							`if (lean_angle_deg <= CRASH_CHECK_ANGLE_MIN_DEG) {`
							`crash_counter = 0;`
							`return;`
							`}`
							
							`// check for angle error over 30 degrees`
							`const float angle_error = attitude_control->get_att_error_angle_deg();`
							`if (angle_error <= CRASH_CHECK_ANGLE_DEVIATION_DEG) {`
								[FIX F3] Split row: attitude-only observation (lean L66, angle-error L73). Velocity observation is the separate row #6 (was wrongly bundled into the L64-73 citation).

6	ArduCopter/crash_check.cpp	78-84	Copter::crash_check()—velocitytest	Continues after the attitude test indicates a possible crash	ahrs.get_velocity_NED(vel_ned) NED speed → magnitude (L80-81)	Resets crash_counter when speed ≥ CRASH_CHECK_SPEED_MAX (10 m/s); otherwise increments toward the 2-second crash trigger disarm. →	`// check for speed under 10m/s (if available)`
							`Vector3f vel_ned;`
							`if (ahrs.get_velocity_NED(vel_ned) && (vel_ned.length() >= CRASH_CHECK_SPEED_MAX)) {`
							`crash_counter = 0;`
							`return;`
							`}`
								[FIX F3] Split row: the velocity observation cited at L78-84 (ahrs.get_velocity_NED L80-81), now accurately centered — not the L64-73 attitude range.

#	File Path	Line(s)	Function/Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

7	ArduPlane/GCS_MAVLink_Plane.cpp	204-212	GCS_MAVLINK_Plane::send_nav_controller_output()	GCS telemetry stream requests NAV_CONTROLLER_OUTPUT	nav_controller navigation state (target bearing, xtrack error, wp distance) — unless in MANUAL	Emits the NAV_CONTROLLER_OUTPUT MAVLink message (navigation-health reporting to the GCS).	`void GCS_MAVLINK_Plane::send_nav_controller_output() const`
							`{`
							`    if (plane.control_mode == &plane.mode_manual) {`
							`        return;`
							`    }`
							`#if HAL_QUADPLANE_ENABLED`
							`    const QuadPlane &quadplane = plane.quadplane;`
							`    if (quadplane.show_vtol_view() && quadplane.using_wp_nav()) {`
							`        const Vector3f &targets = quadplane.attitude_control->get_att_target_euler_cd();`
								INDIRECT/RELATIONAL (navigation). Reporting Sink for navigation state.

8	ArduPlane/Attitude.cpp	604-611	Plane::calc_nav_roll()	Fixed-wing attitude loop computing the roll demand	nav_controller->nav_roll_cd() (L1/navigation lateral roll demand)	Sets nav_roll_cd from the navigation controller, constrained to ±roll_limit_cd.	`  calculate a new nav_roll_cd from the navigation controller`
							` */`
							`void Plane::calc_nav_roll()`
							`{`
							`    int32_t commanded_roll = nav_controller->nav_roll_cd();`
							`    nav_roll_cd = constrain_int32(commanded_roll, -roll_limit_cd, roll_limit_cd);`
							`    update_load_factor();`
							`}`
								INDIRECT/RELATIONAL (navigation). Consumes navigation-controller output (user-cited calc_nav_roll).

9	ArduPlane/navigation.cpp	86-94	Plane::navigate()	10 Hz fixed-wing navigation update	have_position flag (EKF/GPS position validity); next_WP_loc	Returns early (skips navigation) when have_position is false; otherwise computes the desired direction and distance to the next waypoint. Page 35	`void Plane::navigate()`
							`{`
							`    // do not navigate with corrupt data`
							`    // ---------------------------------`
							`    if (!have_position) {`
							`        return;`
							`    }`
							
							`    if (next_WP_loc.lat == 0 && next_WP_loc.lng == 0) {`
								INDIRECT/RELATIONAL (navigation). This is the real symbol behind the user-example 'flight_mode_auto' chain (Table 2a L2 #1); gated on PNT position validity.

10	ArduPlane/navigation.cpp	304-311	Plane::calc_gndspeed_undershoot()	10 Hz GPS/navigation update (Plane::update_GPS_10Hz, ArduPlane/Plane.cpp:490)	ahrs.have_inertial_nav() (inertial-navigation solution availability)	Ground-speed-undershoot (min-groundspeed wind/flyaway compensation) is computed only when inertial nav is available; otherwise it is skipped and left stale/zero.	`    Vector3f velNED;`
							`    if (ahrs.have_inertial_nav() && ahrs.get_velocity_NED(velNED)) {`
							`        const Matrix3f &rotMat = ahrs.get_rotation_body_to_ned();`
							`        Vector2f yawVect = Vector2f(rotMat.a.x,rotMat.b.x);`
							`        if (!yawVect.is_zero()) {`
							`            yawVect.normalize();`
							`            float gndSpdFwd = yawVect * velNED.xy();`
							`            groundspeed_undershoot_is_valid = aparm.min_groundspeed > 0;`
								INDIRECT/RELATIONAL (navigation). User-example PNT-State-Observed accessor ahrs.have_inertial_nav() (libraries/A P_AHRS/AP_AHRS.h:273); the accessor itself is catalogued in Table 2 #3.

Table 5a — Dependency Map (Layer 1 direct edges + Layer 2 transitive chains)

Table 5a: Layer 1 — direct callers / callees with typed dependency

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	Copter::ekf_over_threshold()	ahrs.get_variances(); variance filters; compare g.fs_ekf_thresh	Copter::ekf_check()	Function call	`float position_var, vel_var, height_var, tas_variance;`
					`Vector3f mag_variance;`
					`variances_valid = ahrs.get_variances(vel_var, position_var, height_var, mag_variance, tas_variance);`
					`if (!variances_valid) {`
					`return false;`
					`}`
					`uint32_t now_us = AP_HAL::micros();`

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

2	Plane::ekf_check()	ekf_over_threshold();
		ahrs.request_yaw_reset()
			Plane 10 Hz scheduler	Function call	// compare compass and velocity variance vs threshold
					if (ekf_over_threshold()) {
					// if compass is not yet flagged as bad
					if (!ekf_check_state.bad_variance) {
					// increase counter
					ekf_check_state.fail_count++;
					if (ekf_check_state.fail_count == (EKF_CHECK_ITERATION
					S_MAX-2)) {
					// we are two iterations away from declaring an EK
					F failsafe, ask the EKF if we can reset
					// yaw to resolve the issue
					ahrs.request_yaw_reset();

3	Rover::ekf_over_threshold()	ahrs.get_variances(); compare
		g.fs_ekf_thresh
			Rover::ekf_check() Page	Function call	// use EKF to get variance
					float position_variance, vel_variance, height_variance, tas_va
					riance;
					Vector3f mag_variance;
					ahrs.get_variances(vel_variance, position_variance, height_var
					iance, mag_variance, tas_variance);
					// return true if two of compass, velocity and position varian
					ces are over the threshold
					uint8_t over_thresh_count = 0;
					if (mag_variance.length() >= g.fs_ekf_thresh) {
					over_thresh_count++;

4	Sub::failsafe_ekf_check()	ahrs.get_variances(); AP_HAL::millis();
		gcs().send_text()
			Sub periodic scheduler	Function call	Vector3f magVar;
					float compass_variance;
					float vel_variance;
					ahrs.get_variances(vel_variance, posVar, hgtVar, magVar, tasVa
					r);
					compass_variance = magVar.length();
					if (compass_variance < g.fs_ekf_thresh && vel_variance < g.fs_
					ekf_thresh) {
					last_ekf_good_ms = AP_HAL::millis();
					failsafe.ekf = false;
					AP_Notify::flags.ekf_bad = false;

5	Copter::crash_check() (attitude)	ahrs.cos_roll(); ahrs.cos_pitch(); attitude
		_control->get_att_error_angle_deg()
			Copter::crash_check()	Function call	// check for lean angle over 15 degrees
					const float lean_angle_deg = degrees(acosf(ahrs.cos_roll()*ahr
					s.cos_pitch()));
					if (lean_angle_deg <= CRASH_CHECK_ANGLE_MIN_DEG) {
					crash_counter = 0;
					return;
					}
					// check for angle error over 30 degrees
					const float angle_error = attitude_control->get_att_error_angl
					e_deg();
					if (angle_error <= CRASH_CHECK_ANGLE_DEVIATION_DEG) {

6	Copter::crash_check() (velocity)	ahrs.get_velocity_NED(vel_ned)	Copter::crash_check()	Function call	// check for speed under 10m/s (if available)
					Vector3f vel_ned;
					if (ahrs.get_velocity_NED(vel_ned) && (vel_ned.length() >= CRA
					SH_CHECK_SPEED_MAX)) {
					crash_counter = 0;
					return;
					}

7	GCS_MAVLINK_Plane::send_nav_
	controller_output()
		reads plane.control_mode; nav_controller
		state
			GCS telemetry scheduler	Function call	void GCS_MAVLINK_Plane::send_nav_controller_output() const
					{
					if (plane.control_mode == &plane.mode_manual) {
					return;
					}
					#if HAL_QUADPLANE_ENABLED
					const QuadPlane &quadplane = plane.quadplane;
					if (quadplane.show_vtol_view() && quadplane.using_wp_nav()) {
					const Vector3f &targets = quadplane.attitude_control->get_
					att_target_euler_cd();

8	Plane::calc_nav_roll()	nav_controller->nav_roll_cd();
		update_load_factor()
			Plane fast attitude loop	Function call	calculate a new nav_roll_cd from the navigation controller
					*/
					void Plane::calc_nav_roll()
					{
					int32_t commanded_roll = nav_controller->nav_roll_cd();
					nav_roll_cd = constrain_int32(commanded_roll, -roll_limit_cd,
					roll_limit_cd);
					update_load_factor();
					}

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

9	Plane::navigate()	reads have_position; next_WP_loc; nav_controller update	Plane 10 Hz nav loop Page	Data dependency	void Plane::navigate()
					{
					// do not navigate with corrupt data
					// ---------------------------------
					if (!have_position) {
					return;
					}
					if (next_WP_loc.lat == 0 && next_WP_loc.lng == 0) {

10	Plane::calc_gndspeed_undershoot ()	ahrs.have_inertial_nav();
		ahrs.get_velocity_NED()
			Plane::update_GPS_10Hz() (ArduPlane/Plane.cpp:490)	Function call	Vector3f velNED;
					if (ahrs.have_inertial_nav() && ahrs.get_velocity_NED(velNED))
					{
					const Matrix3f &rotMat = ahrs.get_rotation_body_to_ned();
					Vector2f yawVect = Vector2f(rotMat.a.x,rotMat.b.x);
					if (!yawVect.is_zero()) {
					yawVect.normalize();
					float gndSpdFwd = yawVect * velNED.xy();
					groundspeed_undershoot_is_valid = aparm.min_groundspee
					d > 0;

Table 5a: Layer 2 — full transitive chain, final consumer & hop depth

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	EKF variances (via AP_AHRS)	AP_GPS/EKF → ahrs.get_variances() → ekf_over_threshold() → Copter::ekf_check fail_count → failsafe_ekf_event() → LAND/AltHold	EKF failsafe action (LAND)	5	[DUPLICATED] EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk).
2	EKF variances (Plane)	EKF variances → Plane::ekf_check → bad_variance → EKF failsafe → mode change (RTL/FBWA)	Plane EKF failsafe	4	[DUPLICATED] mirrors #1.
3	EKF variances (Rover)	EKF variances → Rover::ekf_over_threshold → Rover::ekf_check → EKF failsafe action	Rover EKF failsafe	3	[DUPLICATED] mirrors #1.
4	EKF variances (Sub)	EKF variances → Sub::failsafe_ekf_check → failsafe.ekf flag → GCS warn	Sub EKF failsafe flag	3	[DUPLICATED] mirrors #1.
5	AP_AHRS attitude	AHRS attitude → crash_check attitude test → crash_counter → arming.disarm()	motor disarm	3	Attitude branch.
6	AP_AHRS NED velocity	EKF velocity → ahrs.get_velocity_NED() → crash_check speed test → crash_counter → arming.disarm()	motor disarm	4	Velocity branch (F3 split).
7	Navigation controller state	EKF/nav state → nav_controller → send_nav_controller_output → MAVLink → GCS	GCS NAV_CONTROLLER_OUTPUT	4	Reporting branch.
8	Navigation controller roll demand	nav_controller (L1/EKF) → nav_roll_cd() → calc_nav_roll → nav_roll_cd → attitude controller → SRV_Channels	SRV_Channels	5	Roll-demand consumption.
9	EKF position validity	EKF position → have_position → Plane::navigate → nav_controller->update_waypoint → calc_nav_roll → SRV_Channels	SRV_Channels	5	Real symbol behind user-example chain (Table 2a L2 #1).
10	AP_AHRS::have_inertial_nav() (inertial-nav availability)	AP_AHRS::have_inertial_nav() → Plane::calc_gndspeed_undershoot() [ArduPlane/navigation.cpp:305] → Plane::update_GPS_10Hz() [ArduPlane/Plane.cpp:490] Page 38	Plane::update_GPS_10Hz() (groundspeed-undershoot nav compensation)	2	User-example ahrs.have_inertial_nav() observation gating nav groundspeed compensation.

Table 6 — Indirect Timing

#	File Path	Line(s)	Function/Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

1	ArduCopter/failsafe.cpp	37-45	Copter::failsafe_check()	1 kHz timer-failsafe ISR; scheduler tick counter has not advanced since the last call	AP_HAL::micros() (tnow) and AP_Scheduler::ticks() loop counter (timing / loop-cadence state)	When ticks stall and tnow-failsafe_last_timestamp > 2,000,000 us (2 s) sets in_failsafe, drives motors->output_min(), then disarms every 1 s (CPU failsafe).	uint32_t tnow = AP_HAL::micros();
							
							const uint16_t ticks =
							scheduler.ticks();
							if (ticks != failsafe_last_ticks) {
							// the main loop is running, all is
							OK
							failsafe_last_ticks = ticks;
							failsafe_last_timestamp = tnow;
							if (in_failsafe) {
							in_failsafe = false;
								[DUPLICATED] INDIRECT/RELATIONAL (timing). Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk).

#	File Path	Line(s)	Function/Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

2	ArduPlane/failsafe.cpp	22-30	Plane::failsafe_check()	1 kHz timer-failsafe ISR; scheduler tick counter unchanged (main loop stalled)	micros() (tnow) and AP_Scheduler::ticks() (ticks vs last_ticks)	After > 200,000 us (0.2 s) stall sets in_failsafe and passes RC inputs straight to servo outputs (SRV_Channels) every 20 ms instead of disarming.	uint32_t tnow = micros();
							
							const uint16_t ticks = scheduler.ticks();
							if (ticks != last_ticks) {
							// the main loop is running, all is OK
							last_ticks = ticks;
							last_timestamp = tnow;
							in_failsafe = false;
							return;
								[DUPLICATED] INDIRECT/RELATIONAL (timing). Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk). Divergent: 0.2 s trip + RC-passthrough (vs Copter 2 s + disarm).

3	Rover/failsafe.cpp	22-30	Rover::failsafe_check()	1 kHz timer-failsafe ISR; scheduler tick counter unchanged	AP_HAL::micros() (tnow) and AP_Scheduler::ticks() (ticks vs last_ticks)	After > 200,000 us (0.2 s) stall calls arming.disarm(AP_Arming::Method::CPUFAILSAFE).	const uint32_t tnow = AP_HAL::micros();
							
							const uint16_t ticks = scheduler.ticks();
							if (ticks != last_ticks) {
							// the main loop is running, all is OK
							last_ticks = ticks;
							last_timestamp = tnow;
							return;
							}
								[DUPLICATED] INDIRECT/RELATIONAL (timing). Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk).

4	ArduSub/failsafe.cpp	31-39	Sub::mainloop_failsafe_check()	1 kHz timer-failsafe ISR; scheduler tick counter unchanged	AP_HAL::micros() (tnow) and AP_Scheduler::ticks() loop counter	After > 2,000,000 us (2 s) stall sets in_failsafe and drives motors.output_min(). Page 39	uint32_t tnow = AP_HAL::micros();
							
							const uint16_t ticks = scheduler.ticks();
							if (ticks != failsafe_last_ticks) {
							// the main loop is running, all is OK
							failsafe_last_ticks = ticks;
							failsafe_last_timestamp = tnow;
							if (in_failsafe) {
							in_failsafe = false;
								[DUPLICATED] INDIRECT/RELATIONAL (timing). Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk).

5	Blimp/failsafe.cpp	37-45	Blimp::failsafe_check()	1 kHz timer-failsafe ISR; scheduler tick counter unchanged	AP_HAL::micros() (tnow) and AP_Scheduler::ticks() loop counter	After > 2,000,000 us (2 s) stall sets in_failsafe and drives motors->output_min().	uint32_t tnow = AP_HAL::micros();
							
							const uint16_t ticks = scheduler.ticks();
							if (ticks != failsafe_last_ticks) {
							// the main loop is running, all is OK
							failsafe_last_ticks = ticks;
							failsafe_last_timestamp = tnow;
							if (in_failsafe) {
							in_failsafe = false;
								[DUPLICATED] INDIRECT/RELATIONAL (timing). Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk). Blimp included only because it observes scheduler timing state (AAP 0.3.2).

6	ArduCopter/ekf_check.cpp	84-88	Copter::ekf_check()—GCS warn throttle	EKF variance already flagged bad AND 30 s ≥ elapsed since the last warning	AP_HAL::millis() - ekf_check_state.last_warn_time vs EKF_CHECK_WARNI NG_TIME (30 s)	Rate-limits the “EKF variance” GCS_SEVERITY_CRITICAL text to at most once per 30 s; stamps last_warn_time = millis().	// send message to gcs
							if ((AP_HAL::millis() - ekf_check_state.last_warn_time) > EKF_CHECK_WARNING_TIME) {
							gcs().send_text(MAV_SEVERITY_CRITICAL,"EKF variance");
							ekf_check_state.last_warn_time = AP_HAL::millis();
							}
								[DUPLICATED] INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 5 #1 ekf_check]. 30 s warn-throttle copied in ArduPlane/ekf_check.cpp.

#	File Path	Line(s)	Function/Class	Trigger Condition	PNT State Observed	Behavior Change Description	Code Snippet (5-10 lines)	Notes

7	ArduPlane/ekf_check.cpp	78-82	Plane::ekf_check()—GCS warnthrottle	EKF variance already flagged bad AND 30 s ≥ elapsed since the last warning	AP_HAL::millis() - ekf_check_state.last_warn_time vs EKF_CHECK_WARNING_TIME (30 s)	Rate-limits the “EKF variance” GCS text to at most once per 30 s; stamps last_warn_time = millis().	`// send message to gcs`
							`if ((AP_HAL::millis() - ekf_check_state.last_warn_time) > EKF_CHECK_WARNING_TIME) {`
							`gcs().send_text(MAV_SEVERITY_CRITICAL,"EKF variance");`
							`ekf_check_state.last_warn_time = AP_HAL::millis();`
							`}`
								[DUPLICATED] INDIRECT/RELATIONAL (timing). mirrors ArduCopter/ekf_check.cpp:84-88.

8	ArduSub/failsafe.cpp	125-130	Sub::failsafe_ekf_check()—2sconfirmhysteresis	EKF variance bad but not yet bad for a full 2 s	AP_HAL::millis() vs last_ekf_good_ms + 2000 (2 s clock-based confirmation window)	Suppresses the EKF failsafe (clears failsafe.ekf / ekf_bad and returns) until the EKF has been bad for 2 solid seconds, then trips → arming.disarm(EKFFAILSAFE).	`// Bad EKF for 2 solid seconds triggers failsafe`
							`if (AP_HAL::millis() < last_ekf_good_ms + 2000) {`
							`failsafe.ekf = false;`
							`AP_Notify::flags.ekf_bad = false;`
							`return;`
							`}`
								INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 5 #4]. Divergent hysteresis: Sub uses a clock-based 2 s window + 20 s GCS warn-throttle (L141), whereas Copter/Plane use a count-based EKF_CHECK_ITERATIONS_MAX (≈1 s) + 30 s warn-throttle.

9	ArduCopter/ekf_check.cpp	125-132	Copter::ekf_over_threshold()—dtfiltering	Each ekf_over_threshold() evaluation (nominally the 10 Hz ekf_check cadence)	AP_HAL::micros() delta dt = (now_us - → last_ekf_check_us) * 1e-6 (actual inter-call interval)	Applies the measured dt to the position/velocity variance low-pass filters (pos_variance_filt / vel_variance_filt); the effective filter cutoff tracks the real scheduler call interval rather than a fixed rate. Page 40	`uint32_t now_us = AP_HAL::micros();`
							`float dt = (now_us - last_ekf_check_us) * 1e-6f;`
							
							`// always update filtered values as this serves the vibration check as well`
							`position_var = pos_variance_filt.apply(position_var, dt);`
							`vel_var = vel_variance_filt.apply(vel_var, dt);`
							
							`last_ekf_check_us = now_us;`
								INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 5 #1]. EKF check-rate <-> scheduler coupling (AAP 0.5.5): filter behaviour depends on scheduler timing.

10	ArduCopter/fence.cpp	8-16	Copter::fence_checks_async()	1 kHz async fence IO callback	AP_HAL::millis() (now) and AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 40U) (40 ms / 25 Hz gate)	Returns early until 40 ms have elapsed, throttling the geofence breach check to 25 Hz; the position-vs-fence observation itself is Table 4 #3.	`void Copter::fence_checks_async()`
							`{`
							`const uint32_t now = AP_HAL::millis();`
							
							`if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 40U)) { // 25Hz update rate`
							`return;`
							`}`
							
							`if (fence_breaches.have_updates) {`
								[DUPLICATED] INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 4 #3] — timing side of the same function (F1). vs Rover/fence.cpp (divergent rate).

11	Rover/fence.cpp	6-14	Rover::fence_checks_async()	1 kHz async fence IO callback	AP_HAL::millis() (now) and AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 100U) (100 ms / 10 Hz gate)	Returns early until 100 ms have elapsed, throttling the geofence breach check to 10 Hz.	`void Rover::fence_checks_async()`
							`{`
							`const uint32_t now = AP_HAL::millis();`
							
							`if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 100U)) { // 10Hz update rate`
							`return;`
							`}`
							
							`if (fence_breaches.have_updates) {`
								[DUPLICATED] INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 4 #9]. vs ArduCopter/fence.cpp:8-16 — divergent rate (40U/25 Hz vs 100U/10 Hz).

Table 6a — Dependency Map (Layer 1 direct edges + Layer 2 transitive chains)

Table 6a: Layer 1 — direct callers / callees with typed dependency

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

1	Copter::failsafe_check()	AP_HAL::micros(); AP_Scheduler::ticks(); motors->output_min(); motors->armed(false)	failsafe_check_static() <- hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:87)	Function call	`uint32_t tnow = AP_HAL::micros();`
					`const uint16_t ticks = scheduler.ticks();`
					`if (ticks != failsafe_last_ticks) {`
					`// the main loop is running, all is OK`
					`failsafe_last_ticks = ticks;`
					`failsafe_last_timestamp = tnow;`
					`if (in_failsafe) {`
					`in_failsafe = false;`

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

2	Plane::failsafe_check()	micros(); AP_Scheduler::ticks(); rc().read_input(); SRV_Channels::set_output_scaled()	failsafe_check_static() <- hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:103)	Function call	`uint32_t tnow = micros();`
					`const uint16_t ticks = scheduler.ticks();`
					`if (ticks != last_ticks) {`
					`// the main loop is running, all is OK`
					`last_ticks = ticks;`
					`last_timestamp = tnow;`
					`in_failsafe = false;`
					`return;`

3	Rover::failsafe_check()	AP_HAL::micros(); AP_Scheduler::ticks(); arming.disarm(CPUFAILSAFE)	failsafe_check_static() <- hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:111)	Function call	`const uint32_t tnow = AP_HAL::micros();`
					`const uint16_t ticks = scheduler.ticks();`
					`if (ticks != last_ticks) {`
					`// the main loop is running, all is OK`
					`last_ticks = ticks;`
					`last_timestamp = tnow;`
					`return;`
					`}`

4	Sub::mainloop_failsafe_check()	AP_HAL::micros(); AP_Scheduler::ticks(); motors.output_min()	failsafe_check_static() <- hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:71) Page	Function call	`uint32_t tnow = AP_HAL::micros();`
					`const uint16_t ticks = scheduler.ticks();`
					`if (ticks != failsafe_last_ticks) {`
					`// the main loop is running, all is OK`
					`failsafe_last_ticks = ticks;`
					`failsafe_last_timestamp = tnow;`
					`if (in_failsafe) {`
					`in_failsafe = false;`

5	Blimp::failsafe_check()	AP_HAL::micros(); AP_Scheduler::ticks(); motors->output_min()	failsafe_check_static() <- hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:59)	Function call	`uint32_t tnow = AP_HAL::micros();`
					`const uint16_t ticks = scheduler.ticks();`
					`if (ticks != failsafe_last_ticks) {`
					`// the main loop is running, all is OK`
					`failsafe_last_ticks = ticks;`
					`failsafe_last_timestamp = tnow;`
					`if (in_failsafe) {`
					`in_failsafe = false;`

6	Copter::ekf_check() (warn throttle)	AP_HAL::millis(); gcs().send_text()	Copter::ekf_check() bad-variance branch	Function call	`// send message to gcs`
					`if ((AP_HAL::millis() - ekf_check_state.last_warn_`
					`time) > EKF_CHECK_WARNING_TIME) {`
					`gcs().send_text(MAV_SEVERITY_CRITICAL,"EKF var`
					`iance");`
					`ekf_check_state.last_warn_time = AP_HAL::milli`
					`s();`
					`}`

7	Plane::ekf_check() (warn throttle)	AP_HAL::millis(); gcs().send_text()	Plane::ekf_check() bad-variance branch	Function call	`// send message to gcs`
					`if ((AP_HAL::millis() - ekf_check_state.last_warn_`
					`time) > EKF_CHECK_WARNING_TIME) {`
					`gcs().send_text(MAV_SEVERITY_CRITICAL,"EKF var`
					`iance");`
					`ekf_check_state.last_warn_time = AP_HAL::milli`
					`s();`
					`}`

8	Sub::failsafe_ekf_check() (2 s hysteresis)	AP_HAL::millis(); compares last_ekf_good_ms + 2000	Sub periodic scheduler	Function call	`// Bad EKF for 2 solid seconds triggers failsafe`
					`if (AP_HAL::millis() < last_ekf_good_ms + 2000) {`
					`failsafe.ekf = false;`
					`AP_Notify::flags.ekf_bad = false;`
					`return;`
					`}`

9	Copter::ekf_over_threshold() (dt)	AP_HAL::micros(); pos_variance_filt.apply(); vel_variance_filt.apply()	Copter::ekf_check()	Function call	`uint32_t now_us = AP_HAL::micros();`
					`float dt = (now_us - last_ekf_check_us) * 1e-6f;`
					`// always update filtered values as this serves the vibration`
					`check as well`
					`position_var = pos_variance_filt.apply(position_var, dt);`
					`vel_var = vel_variance_filt.apply(vel_var, dt);`
					`last_ekf_check_us = now_us;`

10	Copter::fence_checks_async()	AP_HAL::millis(); AP_HAL::timeout_expired(.,.,40U)	1 kHz async IO process (hal.scheduler register_io_process)	Function call	`void Copter::fence_checks_async()`
					`{`
					`const uint32_t now = AP_HAL::millis();`
					`if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now`
					`, 40U)) { // 25Hz update rate`
					`return;`
					`}`
					`if (fence_breaches.have_updates) {`

Ref #	Component/Function	Directly Calls	Directly Called By	Dependency Type	Code Snippet

11	Rover::fence_checks_async()	AP_HAL::millis();
		AP_HAL::timeout_expired(.,.,100U)
			1 kHz async IO process (hal.scheduler register_io_process)	Function call	`void Rover::fence_checks_async()`
					`{`
					`const uint32_t now = AP_HAL::millis();`
					`if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now`
					`, 100U)) { // 10Hz update rate`
					`return;`
					`}`
					`if (fence_breaches.have_updates) {`

Table 6a: Layer 2 — full transitive chain, final consumer & hop depth

Ref #	PNT Origin	Full Dependency Chain	Final Consumer	Chain Depth	Notes
1	AP_HAL::micros() + AP_Scheduler tick counter	`AP_Scheduler::ticks() / AP_HAL::micros() → failsafe_check_static() [1 kHz timer-failsafe] → Copter::failsafe_check() (>2 s stall) → AP_Motors::output_min()/armed(false)`	AP_Motors (motor min / disarm)	3	[DUPLICATED] Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk).
2	AP_HAL::micros() + AP_Scheduler tick counter	`AP_Scheduler::ticks() / micros() → failsafe_check_static() → Plane::failsafe_check() (>0.2 s stall) → SRV_Channels::set_output_scaled() → servos_output()`	SRV_Channels (RC passthrough)	4	[DUPLICATED] watchdog; divergent 0.2 s trip + RC-passthrough action.
3	AP_HAL::micros() + AP_Scheduler tick counter	`AP_Scheduler::ticks() / micros() → failsafe_check_static() → Rover::failsafe_check() (>0.2 s stall) → AP_Arming::disarm(CPUFAILSAFE)`	AP_Arming (disarm)	3	[DUPLICATED] watchdog.
4	AP_HAL::micros() + AP_Scheduler tick counter	`AP_Scheduler::ticks() / micros() → failsafe_check_static() → Sub::mainloop_failsafe_check() (>2 s stall) → AP_Motors::output_min()`	AP_Motors (motor min)	3	[DUPLICATED] watchdog.
5	AP_HAL::micros() + AP_Scheduler tick counter	`AP_Scheduler::ticks() / micros() → failsafe_check_static() → Blimp::failsafe_check() (>2 s stall) → AP_Motors::output_min()`	AP_Motors (motor min)	3	[DUPLICATED] watchdog; Blimp observes scheduler timing state only (AAP 0.3.2).
6	AP_HAL::millis() wall clock	`AP_HAL::millis() → Copter::ekf_check() (>30 s since last_warn) → gcs().send_text("EKF variance") → GCS_MAVLINK`	GCS_MAVLINK (telemetry text)	3	[DUPLICATED] 30 s warn-throttle (Copter/Plane). [MULTI-CATEGORY w/ Table 5 #1].
7	AP_HAL::millis() wall clock	`AP_HAL::millis() → Plane::ekf_check() (>30 s since last_warn) → gcs().send_text("EKF variance") → GCS_MAVLINK`	GCS_MAVLINK (telemetry text)	3	[DUPLICATED] mirrors #6.
8	AP_HAL::millis() wall clock	`AP_HAL::millis() → Sub::failsafe_ekf_check() (EKF bad ≥ 2 s) → failsafe.ekf = true → arming.disarm(EKFFAILSAFE)`	AP_Arming (EKF failsafe disarm)	3	[MULTI-CATEGORY w/ Table 5 #4]. 2 s confirm + 20 s warn-throttle.
9	AP_HAL::micros() inter-call interval	`AP_HAL::micros() → dt → pos/vel_variance_filt.apply(var, dt) → ekf_over_threshold() → Copter::ekf_check() failsafe decision`	Copter::ekf_check() variance decision	4	EKF check-rate <-> scheduler coupling (AAP 0.5.5). [MULTI-CATEGORY w/ Table 5 #1].
10	AP_HAL::millis() wall clock	`AP_HAL::millis() → AP_HAL::timeout_expired(40U) gate → Copter::fence_checks_async() → AC_Fence::check() → fence_breaches latched → main-loop fence failsafe`	AC_Fence / fence failsafe	5	[DUPLICATED] [MULTI-CATEGORY w/ Table 4 #3]. vs Rover (40U/25 Hz).
11	AP_HAL::millis() wall clock	`AP_HAL::millis() → AP_HAL::timeout_expired(100U) gate → Rover::fence_checks_async() → AC_Fence::check() → fence_breaches latched Page 43`	AC_Fence / fence failsafe	4	[DUPLICATED] [MULTI-CATEGORY w/ Table 4 #9]. vs Copter (100U/10 Hz).

PNT Instance Audit — Current Location → New Service Location

Every Position / Navigation / Timing touch-point inside AP_L1_Control mapped onto the reusable AfsimL1Behavior service (the AHRS state shim, the injected-**dt** timing seam and the **extern "C"** command surface).

Line-number basis. The **Current Location(s)** line numbers below are cited as of the audited HEAD 5b67e27b0a (the pre-refactor baseline), which preserves parity with the AAP 0.6.1 instance table; they are not post-seam current-source line numbers. The extraction's single additive, **default-off** set_update_dt timing seam was applied to AP_L1_Control.cpp / .h after that commit and shifts only those two files' line numbers (it relocates no behavior). Because the seam is **default-off**, when set_update_dt is never called the controller keeps its internal AP_HAL::micros() / millis() timing path unchanged, so existing vehicle callers are unaffected.

PNT Pillar	Behavior	Current Location(s)	Current Accessor	New Service Location
Position	Read vehicle position	AP_L1_Control.cpp:L230, L369	`_ahrs.get_location(_current_loc)`	AHRS shim `get_location()`, fed by `set_state_ne(n, e, ...)`
Navigation (velocity)	Read ground velocity vector	AP_L1_Control.cpp:L236, L375, L507	`_ahrs.groundspeed_vector()`	AHRS shim `groundspeed_vector()`, fed by `set_velocity_EN(velE, velN)`
Navigation (attitude)	Read yaw (radians)	AP_L1_Control.cpp:L59, L61, L402, L536	`_ahrs.get_yaw_rad()`	AHRS shim `get_yaw_rad()`, fed by `set_yaw_cd(yaw_cd)`
Navigation (attitude)	Read yaw (centideg sensor)	AP_L1_Control.cpp:L70, L72, L503, L535	`_ahrs.yaw_sensor`	AHRS shim `yaw_sensor`, fed by `set_yaw_cd(yaw_cd)`
Navigation (attitude)	Read pitch (radians)	AP_L1_Control.cpp:L91	`_ahrs.get_pitch_rad()`	AHRS shim `get_pitch_rad()`, fed by `set_pitch_rad(pitch_rad)`
Navigation (airspeed)	Airspeed scaling factor	AP_L1_Control.cpp:L126, L159	`_ahrs.get_EAS2TAS()`	AHRS shim `get_EAS2TAS()` (injected or unit default)
Timing	Control-step clock	AP_L1_Control.cpp:L214	`AP_HAL::micros()`	`set_update_dt(dt)` injected timebase. Additive and default-off: when `set_update_dt` is never called the controller keeps its internal `AP_HAL::micros()` delta, so existing vehicle callers are unaffected.
Timing	Step delta + state store	AP_L1_Control.cpp:L215, L224	`_last_update_waypoint_us`	Injected-`dt` path (clamp semantics preserved). Additive and default-off: existing callers keep the `_last_update_waypoint_us` `micros()` delta until `set_update_dt` is called.
Timing	Loiter/heading clock	AP_L1_Control.cpp:L448	`AP_HAL::millis()`	Injected time for the loiter path. Additive and default-off: when unset the internal `AP_HAL::millis()` clock is used unchanged.
Navigation (math)	Bearing / NE distance	AP_L1_Control.cpp:L239, L382 / L260, L266, L274, L295, L391	`Location::get_bearing_to` / `get_distance_NE`	Preserved unchanged inside `AP_L1_Control`
Navigation (output)	Roll command	AP_L1_Control.h:L36	`nav_roll_cd()`	`get_roll_deg()` = `nav_roll_cd()/100` → `L1_GetRollDeg`
Navigation (output)	Lateral acceleration	AP_L1_Control.h:L37	`lateral_acceleration()`	`get_lat_accel()` → `L1_GetLatAccel`

Audit-Discipline Flags

Extraction-risk and non-inference discipline markers applied inline in the Notes columns throughout the catalog. Occurrence counts reconcile with the verified corpus totals.

Flag	Occurrences	Meaning
[CIRCULAR]	8	Circular dependency (e.g. AHRS ↔ EKF).
[SHARED-STRUCT]	19	PNT fields packed with non-PNT fields (extraction risk).
[DUPLICATED]	36	Same PNT value read/derived in multiple places.
[MULTI-CATEGORY]	7	Touch-point spans more than one PNT pillar.
[ABSENT]	30	A checked, explicitly-documented absence (non-inference discipline).

Explicitly-documented absences catalogued in the Coverage register: 27.

Coverage & Explicit-Absence Register

Directory-wide token sweeps and explicit absence notations establishing exhaustiveness across libraries/, the four vehicle directories and the .gitmodules boundary edges (10 sub-registers, 109 entries, 27 checked absences).

Indirect supporting-library register (AAP 0.3.1) — AC_Fence / AP_BattMonitor / AP_RCProtocol: non-PNT triggers whose failsafe action is PNT-gated, plus checked absences.

Surface / File	PNT Intersection	Detail	Evidence
libraries/AC_Fence/ (AC_Fence::check + check_fence_polygon / check_fence_circle)	Catalogued ✓	Reads vehicle position vs polygon / circle / altitude fence; position-dependent breach. Fully catalogued in the numbered tables (not an absence).	`libraries/AC_Fence/AC_Fence.cpp:679-843 · Table 4 #4-#8 / Table 4a`
libraries/AP_BattMonitor/ (58 source files; AP_BattMonitor::check_failsafes)	[ABSENT] checked	Directory-wide token sweep found ZERO PNT reads (no ahrs. / gps. / get_location / position_ok / get_velocity_NED). Failsafe trigger is voltage / capacity only; PNT enters at the vehicle action layer, not here.	`libraries/AP_BattMonitor/AP_BattMonitor.cpp:781 (check_failsafes) · action cross-ref ArduCopter/events.cpp:99-121`
libraries/AP_RCProtocol/ (RC failsafe flag)	[ABSENT] checked	Token sweep found ZERO PNT reads. Failsafe is a boolean signal-present flag (failsafe_active); no position / nav / PNT-timing observation. PNT enters at the vehicle action layer.	`libraries/AP_RCProtocol/AP_RCProtocol.h:126-130 (failsafe_active) · action cross-ref ArduCopter/events.cpp:99-121`
ArduCopter/events.cpp (handle_battery_failsafe / radio / GCS FS action select)	PNT-reactive ✓	Non-PNT trigger (battery / RC / GCS) but the chosen failsafe action (RTL / SmartRTL) is gated on position_ok and degrades to LAND when position is invalid — the relational PNT site for batt / RC / GCS failsafe.	`ArduCopter/events.cpp:99-121 (do_failsafe_action BATTERY_FAILSAFE) · gate Table 4 #2`

Cross-vehicle indirect safety-surface register (F16) — Rover / ArduSub failsafe, fence, and mode surfaces, each resolved to its catalogued numbered-table row.

Surface / File	PNT Intersection	Detail	Evidence
Rover/failsafe.cpp (Rover::failsafe_check main-loop watchdog)	Catalogued ✓	Main-loop-lockup watchdog reading scheduler ticks + micros; 0.2 s trip → arming.disarm(CPUFAILSAFE). Catalogued, not an absence.	`Rover/failsafe.cpp:18-31 · Table 6 #3 / Table 6a #3`
Rover/fence.cpp (Rover::fence_checks_async)	Catalogued ✓	Geofence position-vs-boundary check on a 100 ms (10 Hz) timing gate; position-dependent breach. Catalogued.	`Rover/fence.cpp:6-14 · Table 6 #11 / Table 6a #11`
ArduSub/failsafe.cpp (Sub::failsafe_ekf_check)	Catalogued ✓	EKF-variance health watchdog with 2 s confirm + 20 s warn hysteresis. Catalogued.	`ArduSub/failsafe.cpp:100-150 · Table 5 #4 / Table 6 #8`
ArduSub/failsafe.cpp (Sub::mainloop_failsafe_check)	Catalogued ✓	Main-loop-lockup watchdog (scheduler ticks + micros; 2 s motors.output_min). Catalogued. →	`ArduSub/failsafe.cpp:29-55 · Table 6 #4 / Table 6a #4`
ArduSub/mode*.cpp (12 files)	Catalogued ✓	All 12 Sub mode files enumerated with per-file requires_GPS gating and checked absences in the Sub mode-gating matrix below.	`ArduSub/mode.cpp:89-90 (set_mode gate) · Sub mode-gating matrix (this appendix)`

AntennaTracker & Blimp PNT-intersection register (F18; AAP 0.3.2 — included only where they observe / consume PNT): presence sites, per-mode gating, and checked absences.

Surface / File	PNT Intersection	Detail	Evidence
AntennaTracker/tracking.cpp (Tracker::update_vehicle_pos_estimate)	PNT consumer ✓	Reads vehicle.location and vehicle.vel and dead-reckons location_estimate (offset by vel times dt) to point the antenna. Direct PNT consumption (presence, not absence).	`AntennaTracker/tracking.cpp:7-26`
AntennaTracker/mode_auto.cpp + mode_guided.cpp	PNT consumer ✓	Auto / Guided pointing modes consume the tracked vehicle position / bearing to drive pitch and yaw. PNT-reactive.	`AntennaTracker/mode_auto.cpp:5-10 · consumes tracking.cpp:7-26`
AntennaTracker/mode_manual.cpp + mode_scan.cpp + mode_servotest.cpp	[ABSENT] checked	Manual / Scan / Servotest drive the antenna by stick / pattern / test only; token sweep found ZERO vehicle-position reads. Checked absence.	`AntennaTracker/mode_servotest.cpp (0 PNT tokens) · mode_scan.cpp / mode_manual.cpp (0)`
Blimp/mode.cpp (Blimp::set_mode position gate)	Gate enforcer ✓	Rejects PNT modes when position invalid: requires_GPS() and not blimp.position_ok(). Same gate idiom as Copter / Sub.	`Blimp/mode.cpp:78-79`
Blimp/mode_velocity.cpp (ModeVelocity)	Position-gated ✓	requires_GPS()=true needs a valid position / velocity estimate to enter. →	`Blimp/mode.h:181 (requires_GPS=true) · gate Blimp/mode.cpp:78-79`
Blimp/mode_loiter.cpp (ModeLoiter)	Position-gated ✓	requires_GPS()=true position-gated. →	`Blimp/mode.h:226 (requires_GPS=true) · gate Blimp/mode.cpp:78-79`
Blimp/mode_rtl.cpp (ModeRTL)	Position-gated ✓	requires_GPS()=true position-gated. →	`Blimp/mode.h:315 (requires_GPS=true) · gate Blimp/mode.cpp:78-79`
Blimp/mode_manual.cpp (ModeManual)	[ABSENT] checked	requires_GPS()=false no position gate; manual stick control. Checked absence. →	`Blimp/mode.h:138 (requires_GPS=false)`

Surface / File	PNT Intersection	Detail	Evidence
Blimp/mode_land.cpp (ModeLand)	[ABSENT] checked	requires_GPS()=false no horizontal-position gate (descent uses non-PNT references). → Checked absence for horizontal PNT.	Blimp/mode.h:271 (requires_GPS=false)
Blimp/failsafe.cpp (Blimp::failsafe_check watchdog)	Catalogued ✓	Main-loop-lockup watchdog (scheduler ticks + micros; 2 s output_min). Catalogued. →	Blimp/failsafe.cpp:37-45 · Table 6 #5 / Table 6a #5

External-boundary dependency-edge register (AAP 0.4.1 — .gitmodules PNT-relevant vendored submodule boundaries): each boundary resolved to its catalogued PNT edge or publisher site, mirroring the mavlink / DroneCAN edge treatment. Non-inference discipline — every .gitmodules PNT edge is accounted for explicitly, none silently omitted.

Surface / File	PNT Intersection	Detail	Evidence
modules/mavlink (MAVLink message definitions)	Catalogued (edge) ✓	GPS_INPUT ingress (AP_GPS_MAV Source backend) and NAV_CONTROLLER_OUTPUT egress (navigation-health reporting to the GCS). PNT telemetry boundary — catalogued, not an absence.	.gitmodules:7-9 · Table 1 (AP_GPS_MAV.cpp:67-75 handle_msg) · Table 5 (ArduPlane/GCS_MAVLink_Plane.cpp:204-212 send_nav_controller_output)
modules/DroneCAN/DSDL + libcanard (CAN-bus GNSS message definitions)	Catalogued (edge) ✓	DroneCAN / UAVCAN Fix2 GNSS messages decoded into GPS_State. Positioning-source boundary — catalogued, not an absence.	.gitmodules:19-32 · Table 1 (AP_GPS_DroneCAN.cpp:388-396 handle_fix2_msg)
modules/ChibiOS (RTOS low-level clock / tick primitives)	Catalogued (edge) ✓	Board RTOS timer backs the AP_HAL::micros() / millis() monotonic-time primitives. Timing boundary — named explicitly here as the board-timer origin of Table 3a #1.	.gitmodules:13-15 · Table 3 #1 (AP_HAL/system.h:15-20) · Table 3a #1 (RTOS / board-hardware-timer origin)
modules/Micro-XRCE-DDS-Client + modules/Micro-CDR (DDS transport, AP_DDS ROS2) →	PNT publisher (edge) ✓	AP_DDS (AP_DDS_ENABLED=1 by default) reads PNT state and publishes it over the Micro-XRCE-DDS / Micro-CDR transport to ROS2: sensor_msgs/NavSatFix from gps.location() and geographic_msgs/GeoPoseStamped from ahrs.get_location(). PNT-egress boundary — same edge treatment as mavlink / DroneCAN.	.gitmodules:34-40 · libraries/AP_DDS/AP_DDS_Client.cpp:268-270 (NavSatFix gps.location) · :577-584 (GeoPose ahrs.get_location) · ← ← AP_DDS_config.h:11 (#define AP_DDS_ENABLED 1)

ArduPlane vehicle-dir indirect glue register (vehicle-symmetry completeness, AAP 0.3.1 / 0.6.1): ArduPlane failsafe-action and altitude-management glue that reacts to / consumes PNT state, mirroring the catalogued ArduCopter twins.

Surface / File	PNT Intersection	Detail	Evidence
ArduPlane/events.cpp (Plane::handle_battery_failsafe / failsafe_long_on_event / failsafe_short_on_event)	PNT-reactive ✓	Non-PNT trigger (battery / radio / GCS) but the chosen failsafe action is PNT-gated: handle_battery_failsafe reads plane.have_position and plane.current_loc to choose land-sequence continuation vs RTL. Direct Plane analog of the catalogued ArduCopter/events.cpp.	ArduPlane/events.cpp:290-300 (have_position / current_loc land-sequence gate; def @265) · symmetric with ArduCopter/events.cpp:99-121 · gate ArduPlane/mode.cpp:144
ArduPlane/altitude.cpp (Plane::setup_alt_slope / check_home_alt_change)	PNT consumer (Sink) ✓	Positioning Sink: re-reads already-catalogued PNT state — current_loc.get_distance() / line_path_proportion() (AHRS get_location, Table 2) and ahrs.get_home() — to derive the glide slope and altitude target. Consumes, does not produce, PNT.	ArduPlane/altitude.cpp:48-55 (setup_alt_slope current_loc distance) · :31-43 (check_home_alt_change ahrs.get_home) · :109 & :202 (current_loc.alt)

AP_AHRS backend register (Navigation-hub completeness): the AP_AHRS frontend hub is catalogued in Table 2 / 2a; each concrete backend overrides the navigation accessors behind the hub.

Surface / File	PNT Intersection	Detail	Evidence
AP_AHRS_DCM (libraries/AP_AHRS/AP_AHRS_DCM.cpp)	Catalogued ✓	Legacy DCM estimator backend; overrides get_location / groundspeed. Cited in the numbered Navigation tables alongside the hub.	libraries/AP_AHRS/AP_AHRS_DCM.cpp · Table 2 / Table 2a (AHRS hub + DCM)
AP_AHRS_External (libraries/AP_AHRS/AP_AHRS_External.cpp)	PNT backend ✓	External-AHRS backend feeding the hub from an external INS; overrides the position accessor (get_origin). Navigation Source backend behind the AP_AHRS frontend.	libraries/AP_AHRS/AP_AHRS_External.cpp:128 (get_origin) · AP_AHRS_External.h:30 (class : public AP_AHRS_Backend)
AP_AHRS_SIM (libraries/AP_AHRS/AP_AHRS_SIM.cpp)	PNT backend ✓	SITL simulation backend supplying simulated navigation state to the hub; overrides get_location / get_origin. Navigation Source backend behind the AP_AHRS frontend.	libraries/AP_AHRS/AP_AHRS_SIM.cpp:7 (get_location) · :194 (get_origin) · AP_AHRS_SIM.h:37 (class : public AP_AHRS_Backend)

Copter flight modes PNT-gating matrix — 29 files (21 PNT-relevant, 8 checked-absence). Predicate: requires_GPS().

Surface / File	PNT Intersection	Detail	Evidence
mode.cpp (Copter::set_mode() position gate)	Gate enforcer ✓	new_flightmode->requires_GPS() && !copter.position_ok() rejects PNT modes when position invalid	ArduCopter/mode.cpp:303-307 · Table 4 #2
mode_acro.cpp (ModeAcro)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	ArduCopter/mode.h:428 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_acro_heli.cpp (ModeAcro_Heli)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	ArduCopter/mode.h:428 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)

Surface / File	PNT Intersection	Detail	Evidence
mode_althold.cpp (ModeAltHold)	PNT (no horiz-GPS gate) ✓	requires_GPS()=false, but consumes PNT here (pos_control ×9; e.g. altitude pos_control / optical-flow / optional position).	ArduCopter/mode.h:484 · use ArduCopter/mode_althold.cpp:13 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_auto.cpp (ModeAuto)	Position-gated ✓	requires_GPS()=true (conditional: all sub-modes except NAV_ATTITUDE_TIME) — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode_auto.cpp:178 · use ArduCopter/mode_auto.cpp:42 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_autorotate.cpp (ModeAutorotate)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	ArduCopter/mode.h:2047 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_autotune.cpp (ModeAutoTune)	PNT (no horiz-GPS gate) ✓	requires_GPS()=false, but consumes PNT here (pos_control ×5; e.g. altitude pos_control / optical-flow / optional position).	ArduCopter/mode.h:829 · use ArduCopter/mode_autotune.cpp:31 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_avoid_adsb.cpp (ModeAvoidADSB)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1888 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_brake.cpp (ModeBrake)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:854 · use ArduCopter/mode_brake.cpp:13 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_circle.cpp (ModeCircle)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:884 · use ArduCopter/mode_circle.cpp:16 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_drift.cpp (ModeDrift)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:914 · use ArduCopter/mode_drift.cpp:54 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_flip.cpp (ModeFlip)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	ArduCopter/mode.h:942 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_flowhold.cpp (ModeFlowHold)	PNT (no horiz-GPS gate) ✓	requires_GPS()=false, but consumes PNT here (pos_control ×12; e.g. altitude pos_control / optical-flow / optional position).	ArduCopter/mode.h:989 · use ArduCopter/mode_flowhold.cpp:92 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_follow.cpp (ModeFollow)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1918 · use ArduCopter/mode_follow.cpp:33 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_guided.cpp (ModeGuided)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1077 · use ArduCopter/mode_guided.cpp:77 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_guided_custom.cpp (ModeGuidedCustom)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1077 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_guided_nogps.cpp (ModeGuidedNoGPS)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	ArduCopter/mode.h:1253 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_land.cpp (ModeLand)	PNT (no horiz-GPS gate) ✓	requires_GPS()=false, but consumes PNT here (position_ok ×9; e.g. altitude pos_control / optical-flow / optional position).	ArduCopter/mode.h:1277 · use ArduCopter/mode_land.cpp:7 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_loiter.cpp (ModeLoiter)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1323 · use ArduCopter/mode_loiter.cpp:17 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_poshold.cpp (ModePosHold)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1373 · use ArduCopter/mode_poshold.cpp:28 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_rtl.cpp (ModeRTL)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1461 · use ArduCopter/mode_rtl.cpp:21 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_smart_rtl.cpp (ModeSmartRTL)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1577 · use ArduCopter/mode_smart_rtl.cpp:16 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_sport.cpp (ModeSport)	PNT (no horiz-GPS gate) ✓	requires_GPS()=false, but consumes PNT here (pos_control ×9; e.g. altitude pos_control / optical-flow / optional position).	ArduCopter/mode.h:1637 · use ArduCopter/mode_sport.cpp:13 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_stabilize.cpp (ModeStabilize)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	ArduCopter/mode.h:1664 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_stabilize_heli.cpp (ModeStabilize_Heli)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	ArduCopter/mode.h:1664 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_systemid.cpp (ModeSystemId)	PNT (no horiz-GPS gate) ✓	requires_GPS()=false, but consumes PNT here (pos_control ×22; e.g. altitude pos_control / optical-flow / optional position).	ArduCopter/mode.h:1712 · use ArduCopter/mode_systemid.cpp:115 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_throw.cpp (ModeThrow)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1791 · use ArduCopter/mode_throw.cpp:23 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_turtle.cpp (ModeTurtle)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	ArduCopter/mode.h:1849 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)
mode_zigzag.cpp (ModeZigZag)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	ArduCopter/mode.h:1967 · use ArduCopter/mode_zigzag.cpp:75 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)

Plane flight modes PNT-gating matrix — 26 files (20 PNT-relevant, 6 checked-absence). Predicate: does_auto_navigation().

Surface / File	PNT Intersection	Detail	Evidence
mode.cpp (Plane::set_mode() / mode enter)	Gate enforcer ✓	does_auto_navigation() consulted; nav modes consume EKF/AHRS nav state	ArduPlane/mode.cpp:144
mode_LoiterAltQLand.cpp (ModeLoiterAltQLand)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode.h:427 · use ArduPlane/mode_LoiterAltQLand.cpp:8 · gate ArduPlane/mode.cpp:144
mode_acro.cpp (ModeAcro)	[ABSENT] checked	does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	gate ArduPlane/mode.cpp:144
mode_auto.cpp (ModeAuto)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode_auto.cpp:137 · use ArduPlane/mode_auto.cpp:10 · gate ArduPlane/mode.cpp:144
mode_autoland.cpp (ModeAutoLand)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode.h:957 · use ArduPlane/mode_autoland.cpp:79 · gate ArduPlane/mode.cpp:144
mode_autotune.cpp (ModeAutoTune)	[ABSENT] checked	does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	gate ArduPlane/mode.cpp:144
mode_avoidADSB.cpp (ModeAvoidADSB)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (update_loiter ×1; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_avoidADSB.cpp:19 · gate ArduPlane/mode.cpp:144
mode_circle.cpp (ModeCircle)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode.h:399 · use ArduPlane/mode_circle.cpp:7 · gate ArduPlane/mode.cpp:144
mode_cruise.cpp (ModeCruise)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (prev_WP_loc ×4; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_cruise.cpp:83 · gate ArduPlane/mode.cpp:144
mode_fbwa.cpp (ModeFBWA)	[ABSENT] checked	does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	gate ArduPlane/mode.cpp:144
mode_fbwb.cpp (ModeFBWB)	[ABSENT] checked	does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	gate ArduPlane/mode.cpp:144
mode_guided.cpp (ModeGuided)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode.h:360 · use ArduPlane/mode_guided.cpp:11 · gate ArduPlane/mode.cpp:144
mode_loiter.cpp (ModeLoiter)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode.h:427 · use ArduPlane/mode_loiter.cpp:7 · gate ArduPlane/mode.cpp:144
mode_manual.cpp (ModeManual)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (quadplane. ×1; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_manual.cpp:26 · gate ArduPlane/mode.cpp:144
mode_qacro.cpp (ModeQAcro)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (quadplane. ×16; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_qacro.cpp:8 · gate ArduPlane/mode.cpp:144
mode_qautotune.cpp (ModeQAutotune)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (quadplane. ×4; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_qautotune.cpp:11 · gate ArduPlane/mode.cpp:144
mode_qhover.cpp (ModeQHover)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (pos_control ×13; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_qhover.cpp:9 · gate ArduPlane/mode.cpp:144
mode_qland.cpp (ModeQLand)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (quadplane. ×5; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_qland.cpp:9 · gate ArduPlane/mode.cpp:144
mode_qloiter.cpp (ModeQLoiter)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (loiter_nav ×52; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_qloiter.cpp:9 · gate ArduPlane/mode.cpp:144
mode_qrtl.cpp (ModeQRTL)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (quadplane. ×47; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_qrtl.cpp:10 · gate ArduPlane/mode.cpp:144
mode_qstabilize.cpp (ModeQStabilize)	PNT (no horiz-GPS gate) ✓	does_auto_navigation()=false, but consumes PNT here (quadplane. ×19; e.g. altitude pos_control / optical-flow / optional position).	use ArduPlane/mode_qstabilize.cpp:8 · gate ArduPlane/mode.cpp:144
mode_rtl.cpp (ModeRTL)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode.h:507 · use ArduPlane/mode_rtl.cpp:6 · gate ArduPlane/mode.cpp:144
mode_stabilize.cpp (ModeStabilize)	[ABSENT] checked	does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	gate ArduPlane/mode.cpp:144
mode_takeoff.cpp (ModeTakeoff)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode.h:909 · use ArduPlane/mode_takeoff.cpp:74 · gate ArduPlane/mode.cpp:144
mode_thermal.cpp (ModeThermal)	Position-gated ✓	does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.	ArduPlane/mode.h:1025 · use ArduPlane/mode_thermal.cpp:16 · gate ArduPlane/mode.cpp:144
mode_training.cpp (ModeTraining)	[ABSENT] checked	does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	gate ArduPlane/mode.cpp:144

Rover drive modes PNT-gating matrix — 14 files (11 PNT-relevant, 3 checked-absence). Predicate: requires_position(). Page 48

Surface / File	PNT Intersection	Detail	Evidence
mode.cpp (Mode::enter() position gate)	Gate enforcer ✓	rover.ekf_position_ok() && requires_position()/requires_velocity() gate entry	`Rover/mode.cpp:32-38`
mode_acro.cpp (ModeAcro)	PNT (no horiz-GPS gate) ✓	requires_position()=false, but consumes PNT here (wp_nav ×1; e.g. altitude pos_control / optical-flow / optional position).	`Rover/mode.h:235 · use Rover/mode_acro.cpp:37 · gate Rover/mode.cpp:32-38`
mode_auto.cpp (ModeAuto)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`use Rover/mode_auto.cpp:14 · gate Rover/mode.cpp:32-38`
mode_circle.cpp (ModeCircle)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`use Rover/mode_circle.cpp:85 · gate Rover/mode.cpp:32-38`
mode_dock.cpp (ModeDock)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`use Rover/mode_dock.cpp:75 · gate Rover/mode.cpp:32-38`
mode_follow.cpp (ModeFollow)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`use Rover/mode_follow.cpp:12 · gate Rover/mode.cpp:32-38`
mode_guided.cpp (ModeGuided)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`use Rover/mode_guided.cpp:15 · gate Rover/mode.cpp:32-38`
mode_hold.cpp (ModeHold)	[ABSENT] checked	requires_position()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	`Rover/mode.h:630 · gate Rover/mode.cpp:32-38`
mode_loiter.cpp (ModeLoiter)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`use Rover/mode_loiter.cpp:6 · gate Rover/mode.cpp:32-38`
mode_manual.cpp (ModeManual)	[ABSENT] checked	requires_position()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	`Rover/mode.h:681 · gate Rover/mode.cpp:32-38`
mode_rtl.cpp (ModeRTL)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`use Rover/mode_rtl.cpp:11 · gate Rover/mode.cpp:32-38`
mode_simple.cpp (ModeSimple)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`gate Rover/mode.cpp:32-38`
mode_smart_rtl.cpp (ModeSmartRTL)	Position-gated ✓	requires_position()=true — PNT position/nav required to enter; behaviour gated.	`use Rover/mode_smart_rtl.cpp:17 · gate Rover/mode.cpp:32-38`
mode_steering.cpp (ModeSteering)	[ABSENT] checked	requires_position()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	`Rover/mode.h:785 · gate Rover/mode.cpp:32-38`

Sub flight modes PNT-gating matrix — 12 files (7 PNT-relevant, 5 checked-absence). Predicate: requires_GPS().

Surface / File	PNT Intersection	Detail	Evidence
mode.cpp (Sub::set_mode() position gate)	Gate enforcer ✓	new_flightmode->requires_GPS() && !sub.position_ok() rejects PNT modes	`ArduSub/mode.cpp:89-90`
mode_acro.cpp (ModeAcro)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	`ArduSub/mode.h:155 · gate ArduSub/mode.cpp:89-90`
mode_althold.cpp (ModeAlthold)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	`ArduSub/mode.h:201 · gate ArduSub/mode.cpp:89-90`
mode_auto.cpp (ModeAuto)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	`ArduSub/mode.h:314 · use ArduSub/mode_auto.cpp:11 · gate ArduSub/mode.cpp:89-90`
mode_circle.cpp (ModeCircle)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	`ArduSub/mode.h:382 · use ArduSub/mode_circle.cpp:10 · gate ArduSub/mode.cpp:89-90`
mode_guided.cpp (ModeGuided)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	`ArduSub/mode.h:260 · use ArduSub/mode_guided.cpp:34 · gate ArduSub/mode.cpp:89-90`
mode_manual.cpp (ModeManual)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	`ArduSub/mode.h:132 · gate ArduSub/mode.cpp:89-90`
mode_motordetect.cpp (ModeMotordetect)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	`ArduSub/mode.h:427 · gate ArduSub/mode.cpp:89-90`
mode_poshold.cpp (ModePoshold)	Position-gated ✓	requires_GPS()=true — PNT position/nav required to enter; behaviour gated.	`ArduSub/mode.h:355 · use ArduSub/mode_poshold.cpp:13 · gate ArduSub/mode.cpp:89-90`
mode_stabilize.cpp (ModeStabilize)	[ABSENT] checked	requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).	`ArduSub/mode.h:178 · gate ArduSub/mode.cpp:89-90`
mode_surface.cpp (ModeSurface)	PNT (no horiz-GPS gate) ✓	requires_GPS()=false, but consumes PNT here (wp_nav ×1; e.g. altitude pos_control / optical-flow / optional position).	`ArduSub/mode.h:404 · use ArduSub/mode_surface.cpp:54 · gate ArduSub/mode.cpp:89-90`
mode_surftrak.cpp (ModeSurftrak)	PNT (no horiz-GPS gate) ✓	requires_GPS()=false, but consumes PNT here (pos_control ×3; e.g. altitude pos_control / optical-flow / optional position).	`ArduSub/mode.h:201 · use ArduSub/mode_surftrak.cpp:91 · gate ArduSub/mode.cpp:89-90`